

Table of Contents

	Page Numbers
1. Introduction	3
1.1 Purpose	3
1.2 Intended Customers/Users	3
1.3 Product Scope	3
1.4 Outline of the document	4
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions	5
2.3 User Classes	6
2.4 Design and Implementation Constraints	7
3. Diagrams	8
3.1 UML Diagrams	8
3.2 ER Diagrams	31
3.3 DFD Diagrams	32
4. External Interface Requirements/Screenshots	33
4.1 User Interfaces	33
4.2 Hardware Interfaces	33
4.3 Software Interfaces	33
4.4 Communication Interfaces	33
5. Other Non-Functional Requirements	34
5.1 Performance Requirements	34
5.2 Safety Requirements	34
5.3 Security Requirements	34
5.4 Software Quality Attributes	34
5.5 Business Policies/Rules (if any)	35
6. Testing	36
6.1 White-Box Testing	36
6.2 Black-Box Testing	40
7. Conclusion	48
8. References	48
9. Acknowledgements	48

1. Introduction

1.1 Purpose

- The goal of this project is to create a Digital Lost and Found System that helps people easily report, search for, and recover lost items.
- Handling missing items is done manually.
- The system also purports to ease the process by creating an online system for people to report lost and found items, as well as monitor their status.
- Administrator verification is introduced to ensure safety and the prevention of fraud claims.
- The document comprises information on the working of the system, its specifications, limitations, as well as its utilization by the users as well as the administrators.

1.2 Intended Customers / Users

- **Normal User** - These are general users who can misplace or find things within public or shared areas. These users make use of the system to lose things, find things, as well as search.
- **Found Item Reporter** - Users who find lost articles belonging to other users. They are responsible for reporting articles they find so they can be returned to their rightful owner.
- **System Supervisor** - One who is entrusted to check on a daily basis for system activities and report to the administrator, if necessary, for corrective measures to be taken.
- **Administrator** - This is an authorized person in charge of the system, responsible for monitoring and maintaining control of the system. This person also checks reports and verifies claims, as well as ensuring system security.

1.3 Product Scope

- The Digital Lost and Found System is intended to be a web-based platform that enables users to log lost and found objects under a simplified and organized system.
- The system allows various user roles including normal users, students, item finders, and administrators.
- The users are able to enter information about items, search for items, and check the status of their reports.
- Verification by the administrator ensures security and helps to overcome the issue of false.
- A systematic process for item reporting and successful item recovery from an item recovery system is provided by the final system.

1.4 Outline of the Document

➤ Introduction :-

Introduces the project, its purpose, intended users, product scope, and the structure of the document.

➤ Overall Description :-

Provides a high-level overview of the system, including its environment, major functions, user classes, and constraints.

➤ Diagrams :-

Presents UML, ER, and DFD diagrams along with assumptions and dependencies to visually explain the system design.

➤ External Interface Requirements/Screenshots :-

Describes user interfaces, hardware and software interfaces, and communication requirements of the system.

➤ Other Non-Functional Requirements :-

Explains performance, safety, security, quality attributes, and business rules of the system.

➤ Testing :-

Describes the testing methods used, including black-box and white-box testing.

➤ Conclusion :-

Summarizes the project and highlights its outcomes and benefits.

➤ References :-

Lists the books, websites, and resources referred to during project development.

➤ Acknowledgements :-

Expresses gratitude to individuals and institutions that supported the project.

2. Overall Description

2.1 Product Perspective

- The Digital Lost and Found System is a web-based application.
- It runs in environments like colleges, hostels, offices, and public places.
- The system is not a standalone system; it requires:
 - Internet.
 - A database for storing user and item information.
- It can be used in conjunction with a pre-existing user authentication system for authenticating users.
- The system interacts with:
 - Database Management System for storing lost items, found items, and claims.
- The system neither communicates with payment gateways nor with hardware sensors.
- All verification and approval tasks are carried out by the administrator.

2.2 Product Functions

➤ **User Registration**

Allows users to create an account by providing basic details to access the system.

➤ **User Login and Logout**

Enables registered users and administrators to securely log in and log out of the system.

➤ **Report Lost Item**

Allows users to submit details of items they have lost, such as item name, description, location, and date.

➤ **Report Found Item**

Allows users to report items they have found so they can be matched with lost item reports.

➤ **Search Items**

Enables users to search for lost or found items using keywords or categories.

➤ **View Item Details**

Allows users to view item information while keeping personal details hidden until verification.

➤ **Claim Item**

Enables users to send a request to claim an item by providing ownership details.

➤ **Administrator Verification**

Allows the administrator to verify lost and found reports and approve or reject claims.

➤ **Status Tracking**

Shows the current status of items such as lost, found, under verification, or returned.

2.3 User Classes

➤ **Normal User (Item loser)**

Access Levels:

- Register and login.
- Report lost items.
- Search lost and found items.
- View item details.
- Submit claim requests.

➤ **Found Item Reporter**

Access Levels:

- Register and login.
- Report found items.
- View basic item information.
- Respond through the system after admin approval.

➤ **Administrator**

Access Levels:

- Login with admin privileges.
- Verify lost and found reports.
- Approve or reject claim requests.
- Manage users and item records.
- Update item status.
- Generate reports

➤ **System Supervisor**

Access Levels:

- Maintain system performance.
- Manage backups and data security, Handle system updates.

2.4 Design and Implementation Constraints

➤ Hardware Constraints

- Basic computing devices are needed: desktops, laptops, or smartphones.
- Performance may be affected on devices with low memory or processing power.

➤ Software Requirements

- The system requires a current web browser to function as designed.
- The server side technologies and database system should be installed.

➤ Regulatory Constraints

- User data, such as name and contact details, should be used responsibly.
- Basic rules regarding data privacy need to be adhered to so that the misused of personal information is not done.

➤ Network Dependency

- The system will demand that at least there is an active Internet connection.
- Limited or no network connectivity will restrict access to system features.

➤ Database Constraints

- The system relies on a database to store user and item information.
- Non-optimized very large volumes of data can impact the performance.

➤ Security Constraints

- Most functionalities require a user authentication.
- Sensitive information is hidden until administrator verification.
- System is not compatible with enhanced security mechanisms like biometric authentications.

3. Diagrams

3.1 UML Diagrams

A. USE CASE DIAGRAMS

Module 1 :- User Management

➤ **Description :-**

The User Management module is responsible for handling user authentication and access control. It allows users to create a new account, log in to the system, and log out securely after completing their activities.

➤ **Actors :-**

- **Primary Actor:** User

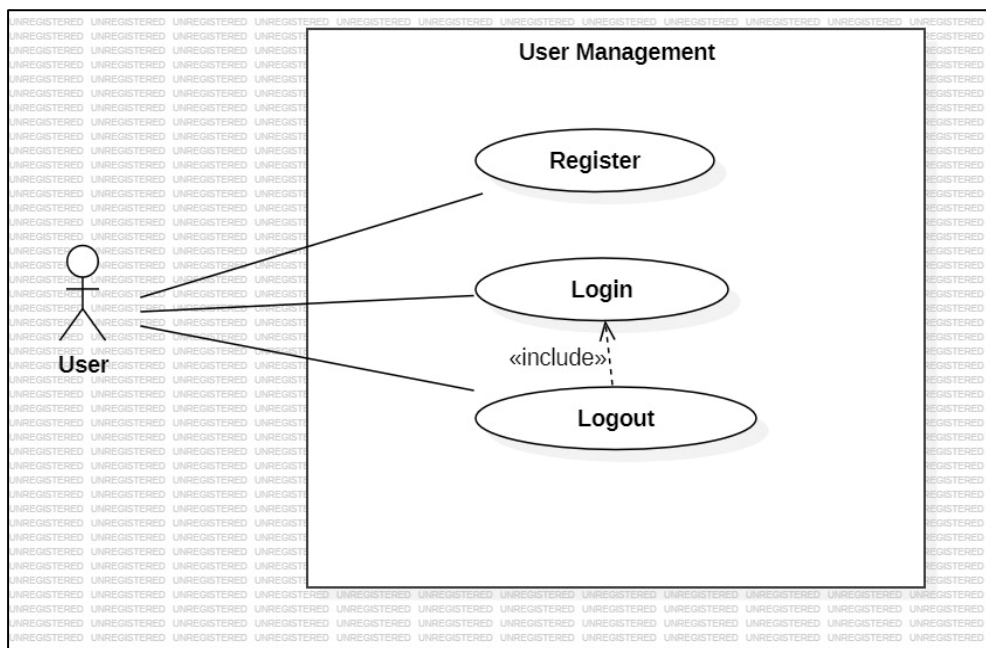
➤ **Use Cases :-**

- Register
- Login
- Logout

➤ **Relationships :-**

- New users must register before logging in.
- Only logged-in users can log out.

➤ **Use Case Diagram :-**



Module 2 :- Lost Item Management

➤ Description :-

The Lost Item Management module allows users to report items they have lost. Users can also view, edit, or delete their submitted lost item reports whenever required.

➤ Actors :-

- **Primary Actor:** User

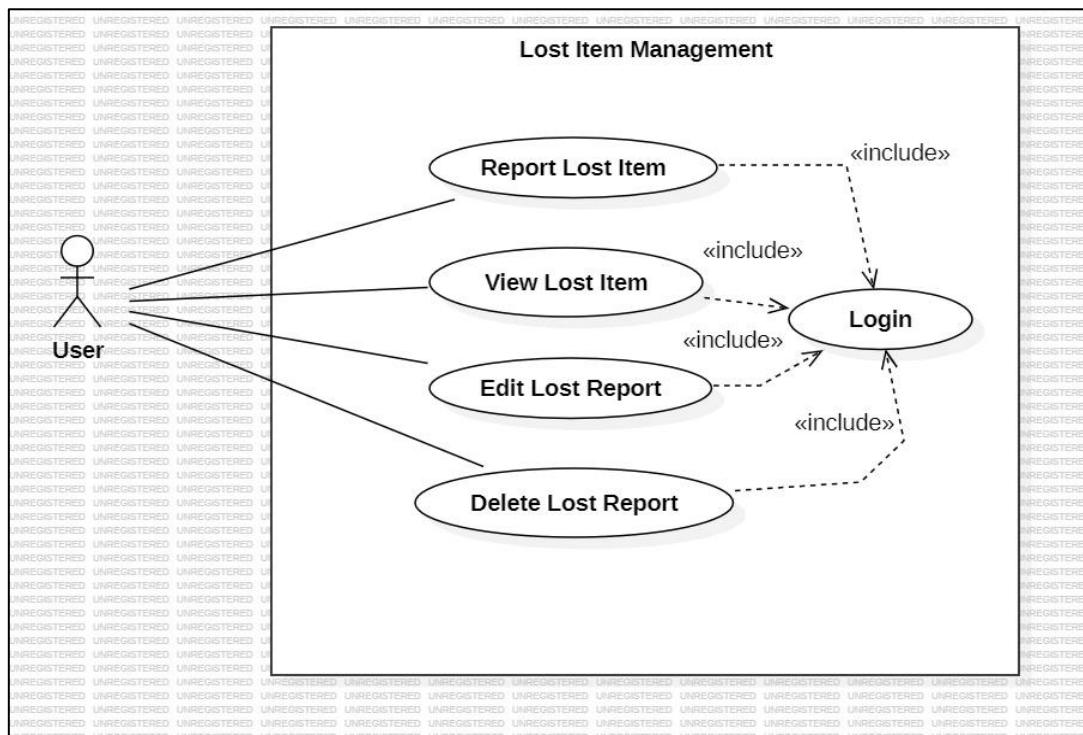
➤ Use Cases :-

- Report Lost Item
- View Lost Item
- Edit Lost Report
- Delete Lost Report

➤ Relationships :-

- All the above operations include Login to ensure that only authenticated users can report or manage lost item details.

➤ Use Case Diagram :-



Module 3 :- Found Item Management

➤ Description :-

The Found Item Management module enables users to report items they have found. Users can manage their found item reports by viewing, editing, or deleting them as needed.

➤ Actors :-

- **Primary Actor:** User

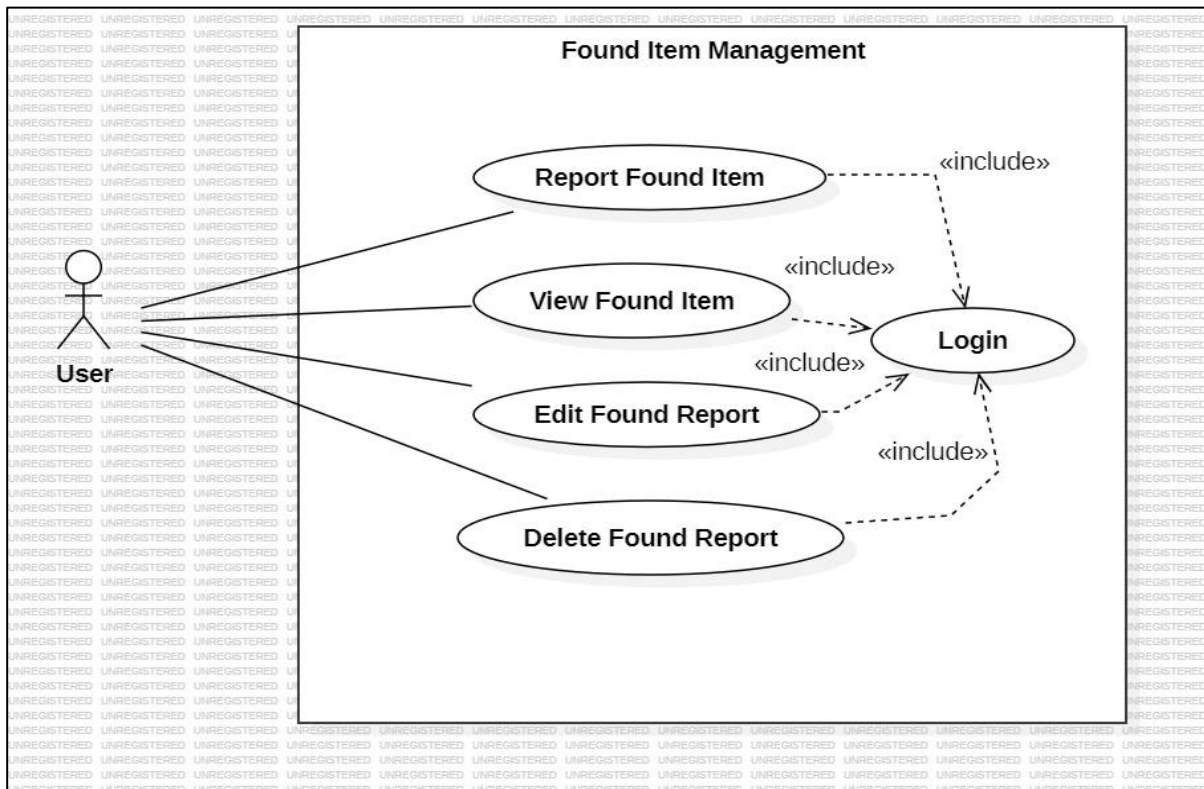
➤ Use Cases :-

- Report Found Item
- View Found Item
- Edit Found Report
- Delete Found Report

➤ Relationships :-

- Each use case includes Login, ensuring that only authorized users can access and manage found item information.

➤ Use Case Diagram :-



Module 4 :- Search and Matching

➤ Description :-

The Search and Matching module helps users search for lost and found items available in the system. It also provides options to filter results, view item details, and match lost items with found items.

➤ Actors :-

- **Primary Actor:** User

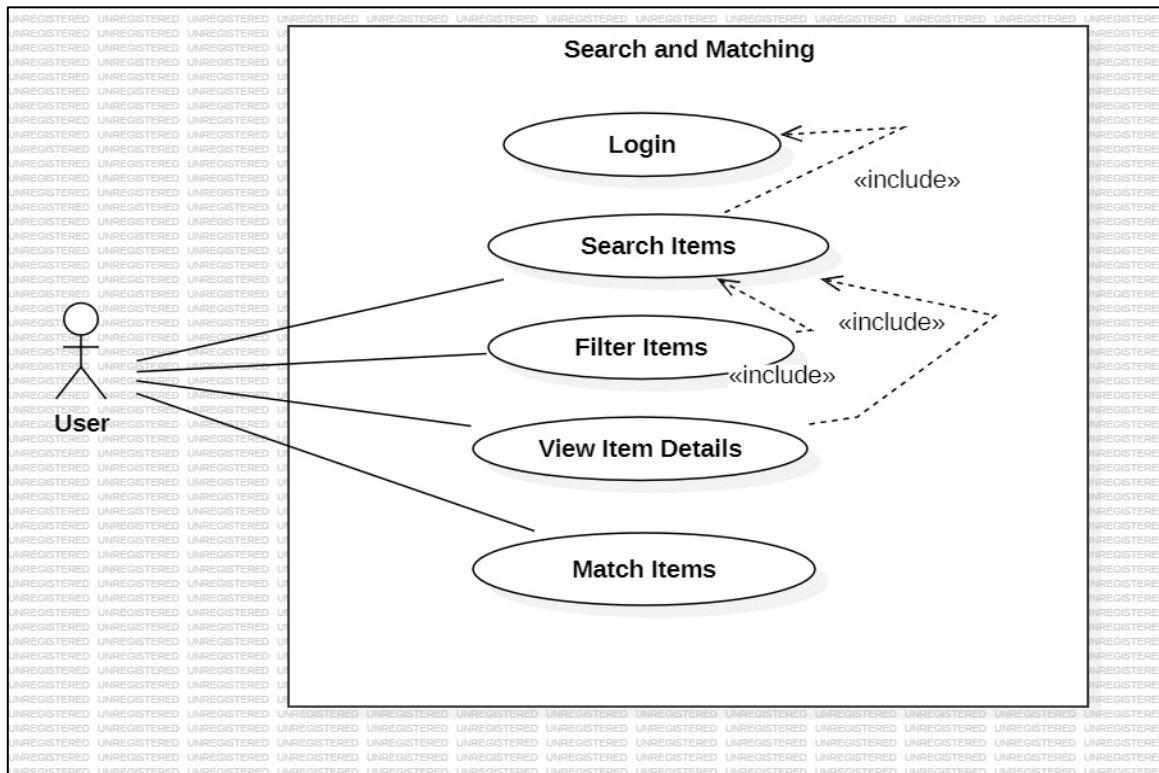
➤ Use Cases :-

- Search Items
- Filter Items
- View Item Details
- Match Items

➤ Relationships :-

- All search-related activities include Login for secure system access.
- Filter Items and View Item Details depend on the Search Items use case, as users must search before applying filters or viewing details.

➤ Use Case Diagram :-



Module 5 :- Admin Verification

➤ Description :-

The Admin Verification module allows administrators to verify the authenticity of lost and found item reports submitted by users. This helps maintain the reliability and accuracy of the system.

➤ Actors :-

- **Primary Actor:** Administrator

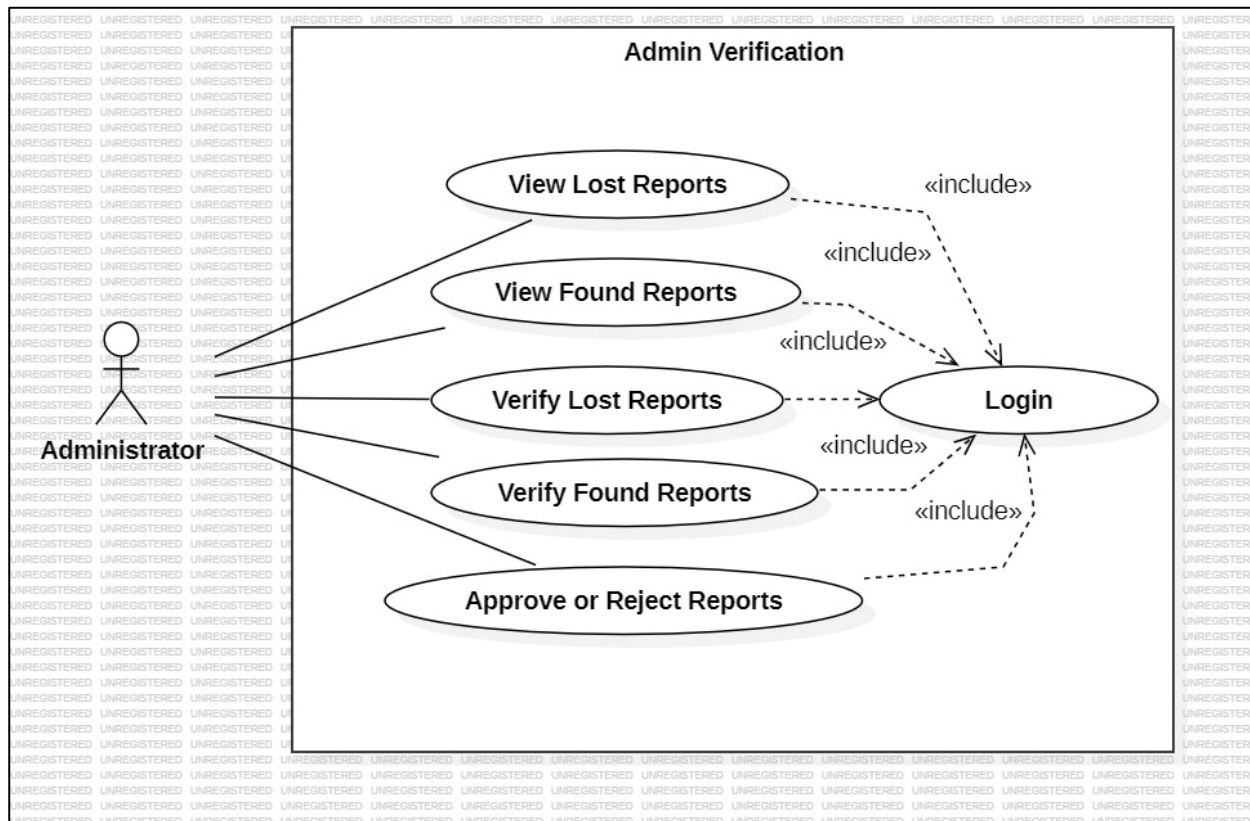
➤ Use Cases :-

- View Lost Reports
- View Found Reports
- Verify Lost Reports
- Verify Found Reports
- Approve or Reject Reports

➤ Relationships :-

- All administrative verification activities include Login to ensure secure access for administrators only.

➤ Use Case Diagram :-



Module 6 :- Notification and Claim Management

➤ Description :-

The Notification and Claim Management module manages notifications and claim processes after items are successfully matched and verified. It ensures smooth communication between users and administrators.

➤ Actors :-

- **Primary Actors:** User, Administrator

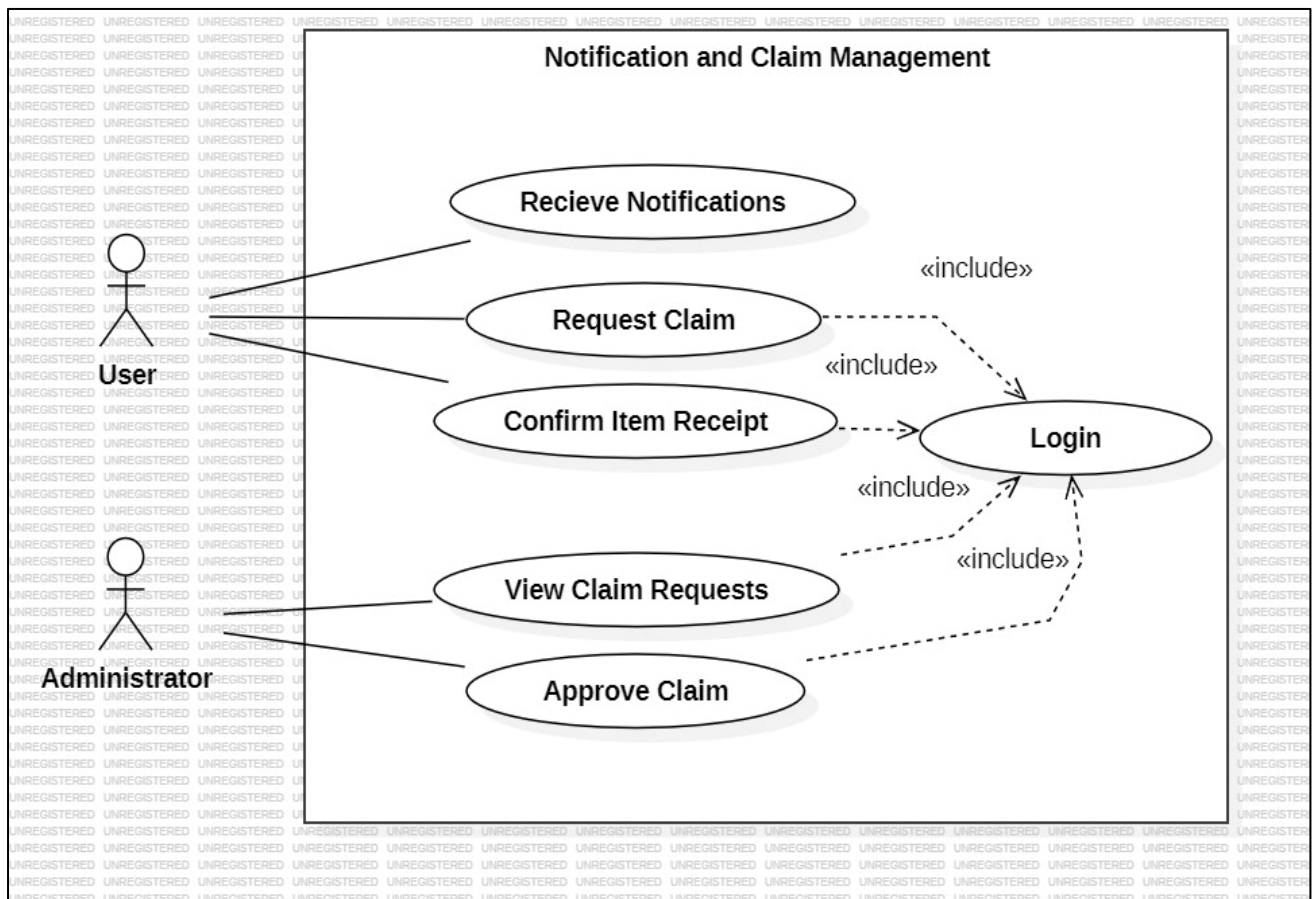
➤ Use Cases :-

- Receive Notifications (User)
- Request Claim (User)
- Confirm Item Receipt (User)
- View Claim Requests (Administrator)
- Approve Claim (Administrator)

➤ Relationships :-

- All notification and claim-related operations include Login to ensure proper authentication and authorization.

➤ Use Case Diagram :-



B. CLASS DIAGRAMS

Module 1 :- User Management

➤ Description :-

This module handles user registration, login, and logout functionalities. It authenticates users and supports password recovery when needed. It ensures secure and controlled access to the system.

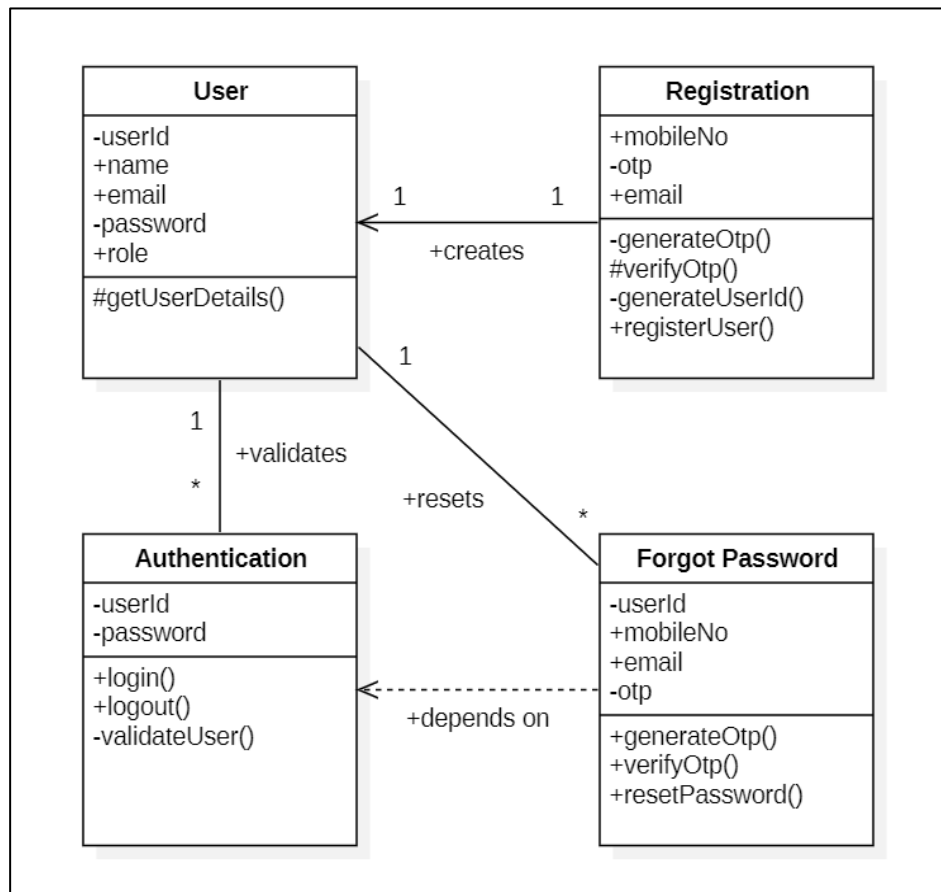
➤ Actors :-

- **Primary Actors:** User

➤ Classes :-

- User
- Registration
- Authentication
- Forgot Password

➤ Class Diagram :-



Module 2 :- Lost Item Management

➤ Description :-

This module allows users to report items that they have lost. It stores lost item details such as location, date, and current status. It helps in tracking and managing lost items efficiently.

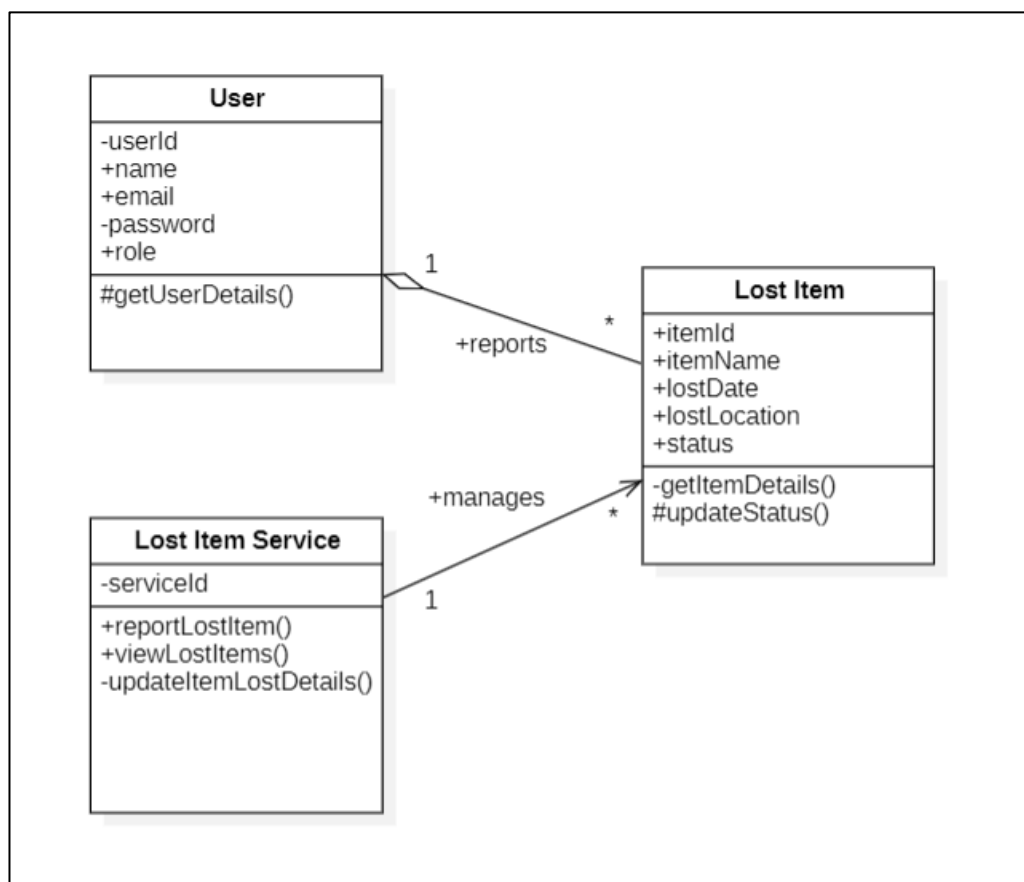
➤ Actors :-

- **Primary Actors:** User

➤ Classes :-

- User
- Lost Item
- Lost Item Service

➤ Class Diagram :-



Module 3 :- Found Item Management

➤ Description :-

This module enables users to report items they have found. It stores found item information for reference and matching purposes. It helps organize and manage found item records.

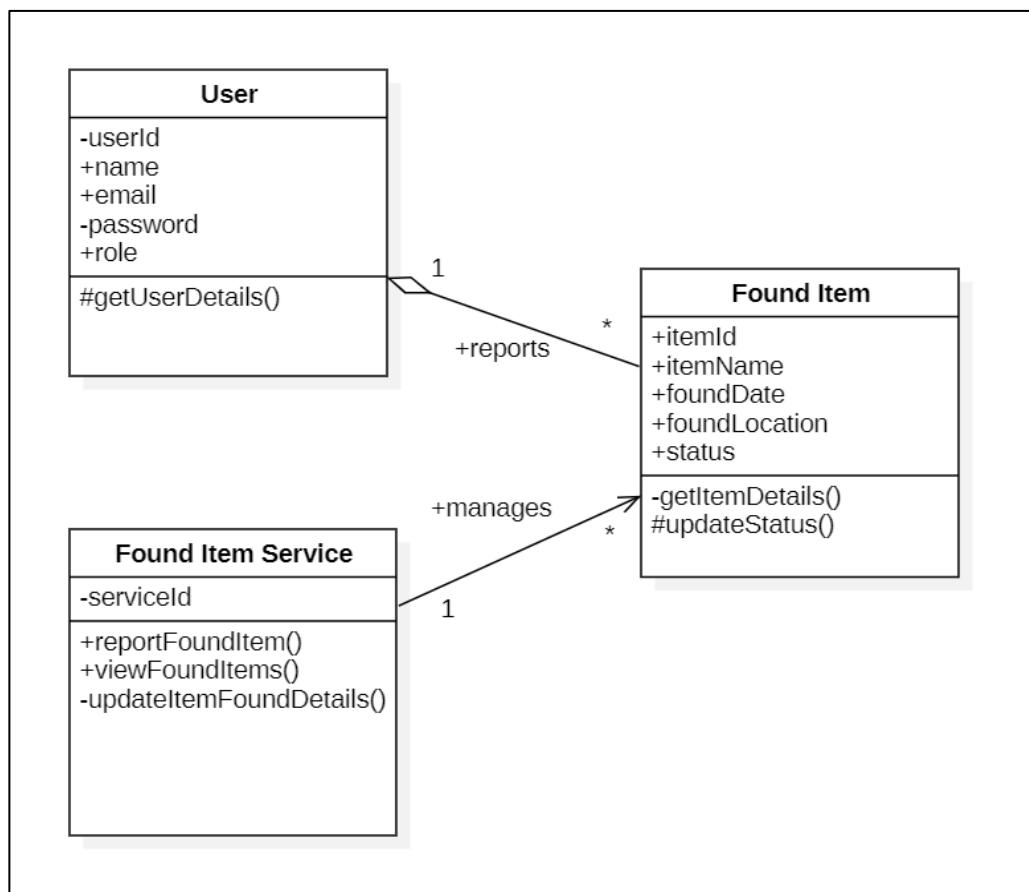
➤ Actors :-

- **Primary Actors:** User

➤ Classes :-

- User
- Found Item
- Found Item Service

➤ Class Diagram :-



Module 4 :- Searching and Matching

➤ Description :-

This module provides search functionality for both lost and found items. It compares item details to identify possible matches. It improves the chances of recovering lost items.

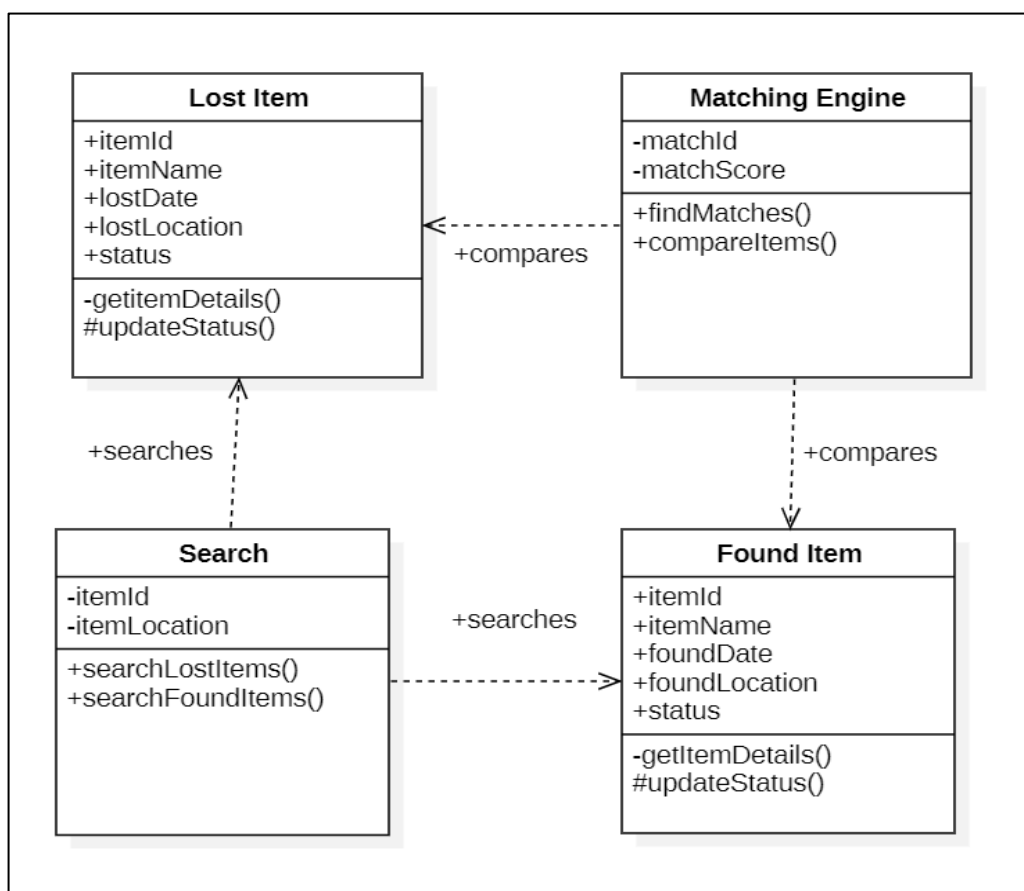
➤ Actors :-

- **Primary Actors:** User

➤ Classes :-

- Lost Item
- Matching Engine
- Found Item
- Search

➤ Class Diagram :-



Module 5 :- Admin Verification

➤ Description :-

This module allows users to submit claims for lost items. Administrators verify the claims and approve or reject them accordingly. It ensures that items are returned to the rightful owners.

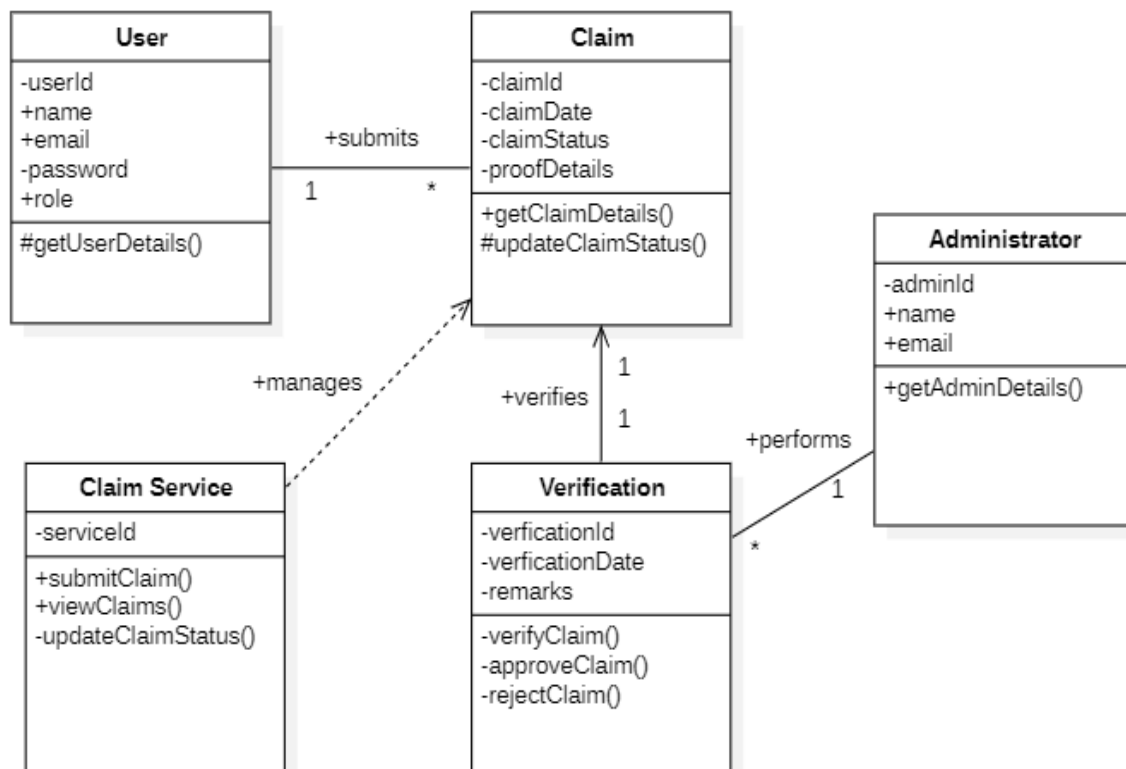
➤ Actors :-

- **Primary Actors:** User, Administrator

➤ Classes :-

- User
- Claim
- Claim Service
- Verification
- Administrator

➤ Class Diagram :-



Module 6 :- Notification and Claim Management

➤ Description :-

This module allows administrators to manage users and monitor system activities. It provides access to reports and item status updates. It ensures smooth and reliable operation of the system.

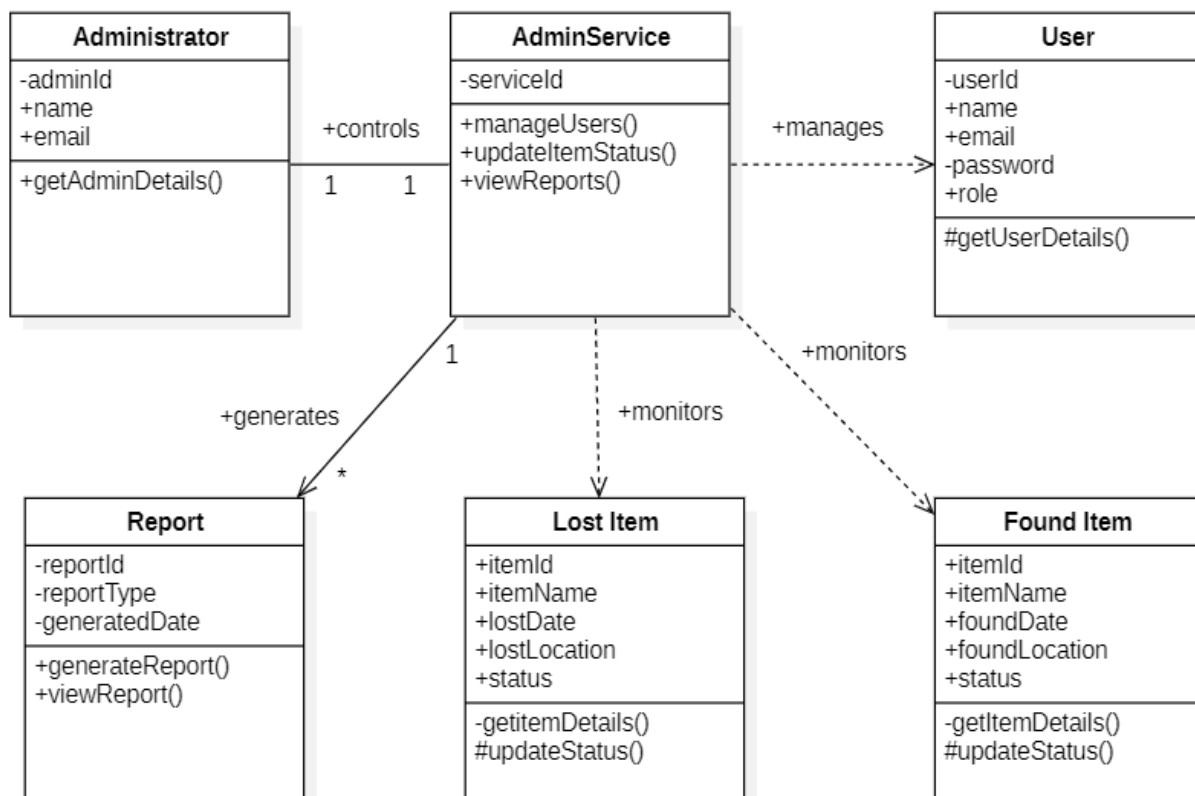
➤ Actors :-

- **Primary Actors:** User, Administrator

➤ Classes :-

- User
- Report
- Lost Item
- Found Item
- Administrator
- AdminService

➤ Class Diagram :-



C. State Diagrams

Module :- Admin Verification

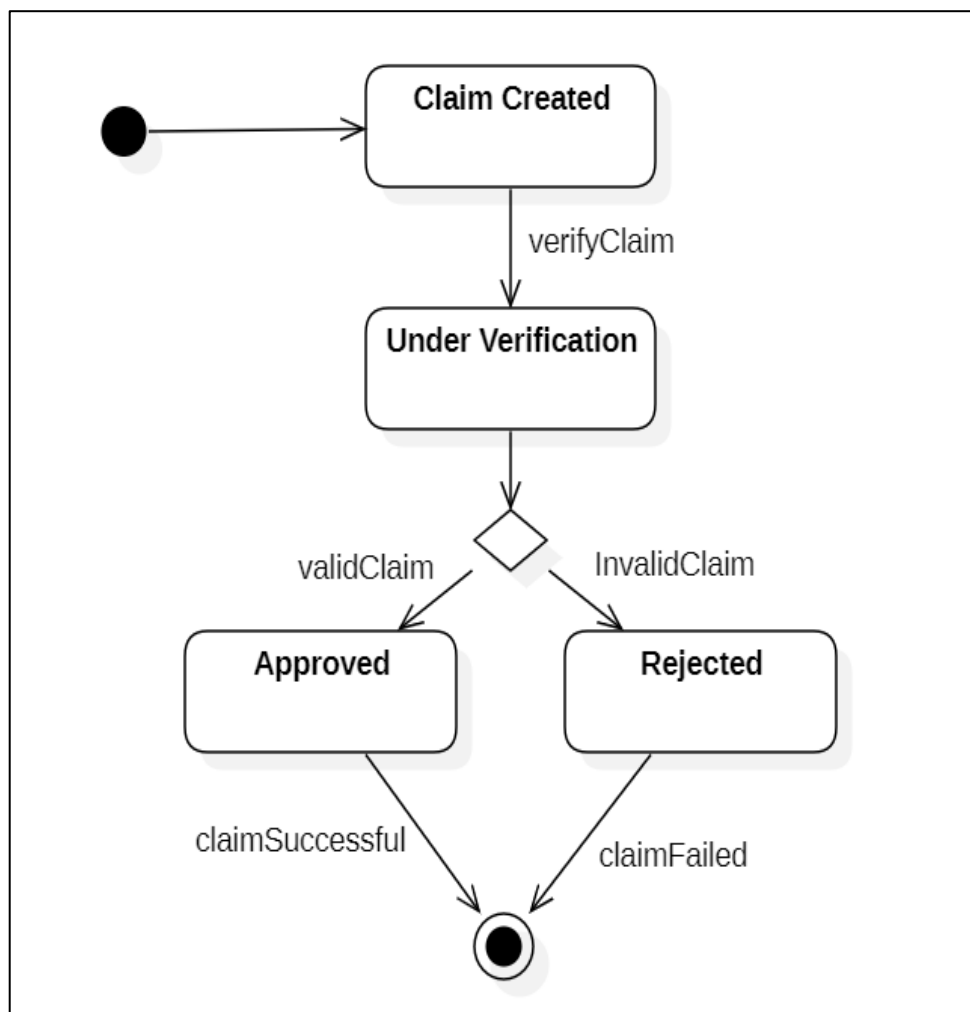
1. Claim Object

➤ Description :-

This state diagram explains how a claim is handled in the system. The user submits a claim, which is then checked by the administrator. After verification, the claim is either approved or rejected, and the process ends.

➤ States Involved :-

- Claim Created
- Under Verification
- Approved
- Rejected



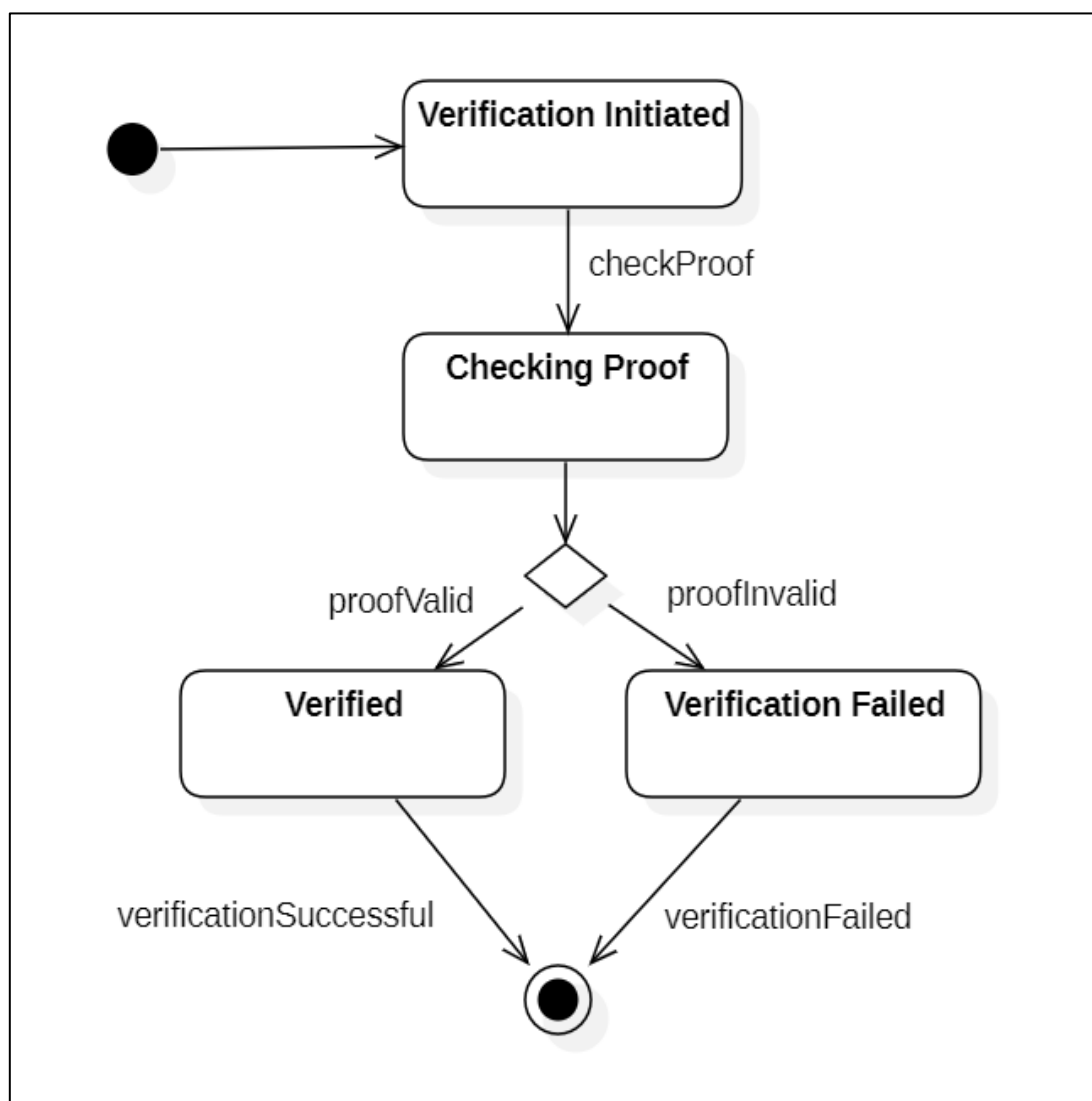
2. Verification Object

➤ Description :-

This state diagram describes how the verification process is carried out in the system. The administrator reviews the claim and checks the provided proof. Based on the verification result, the claim is either verified successfully or marked as failed.

➤ States Involved :-

- Verification Initiated
- Checking Proof
- Verified
- Verification Failed



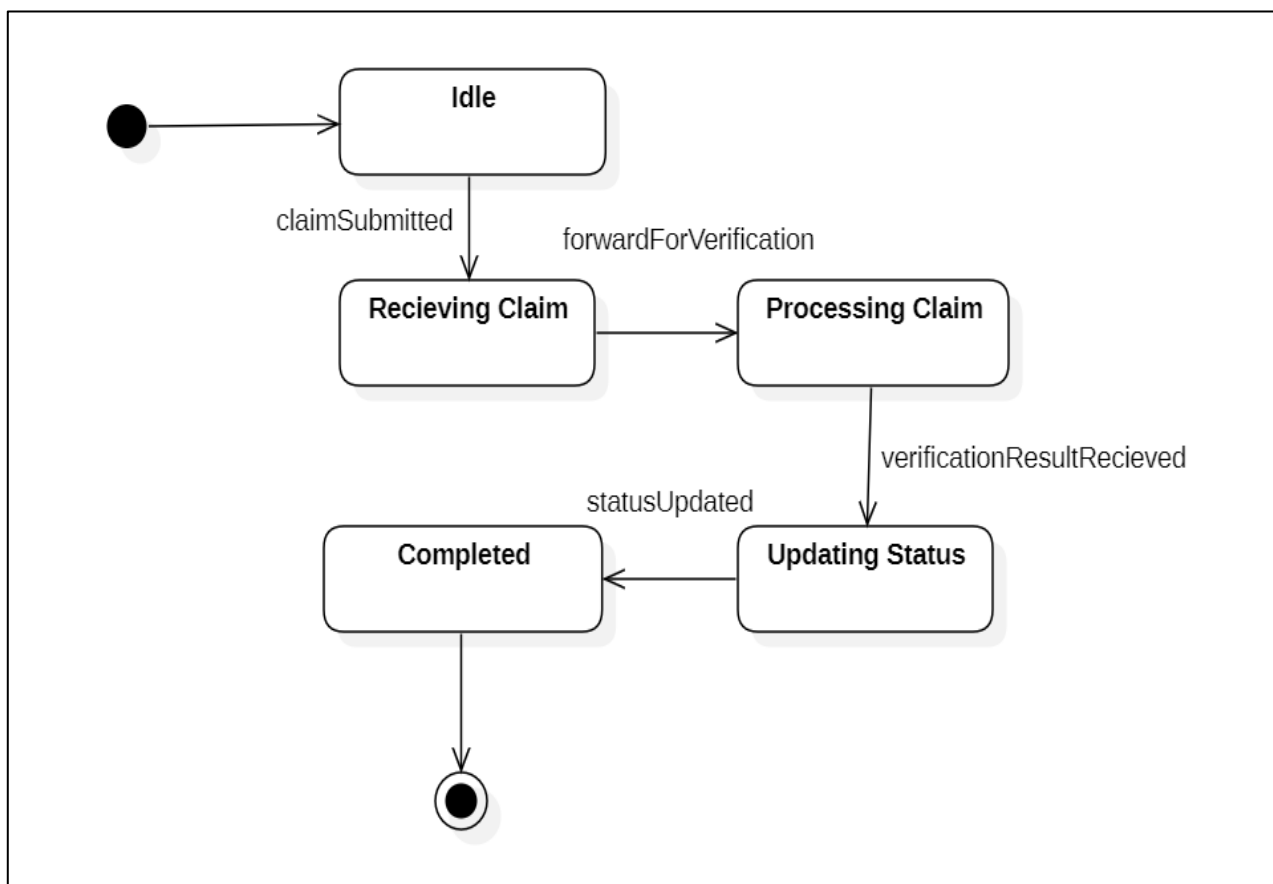
3. Claim Service Object

➤ Description :-

This state diagram explains how a claim is handled in the system. The user submits a claim, which is then checked by the administrator. After verification, the claim is either approved or rejected, and the process ends.

➤ States Involved :-

- Idle
- Receiving Claim
- Processing Claim
- Updating Status
- Completed

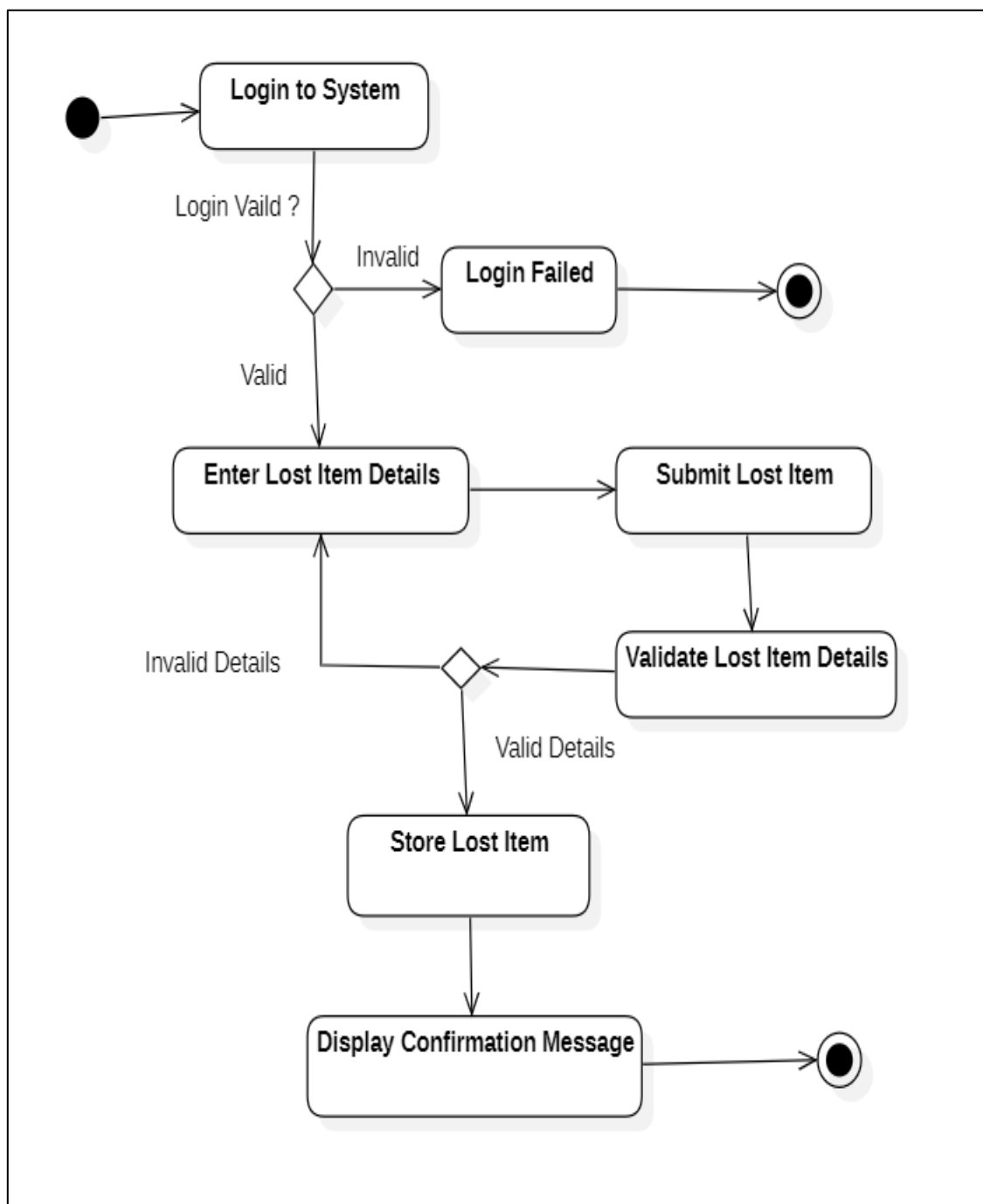


D. Activity Diagrams

Module 2 :- Lost Item Management

➤ Description :-

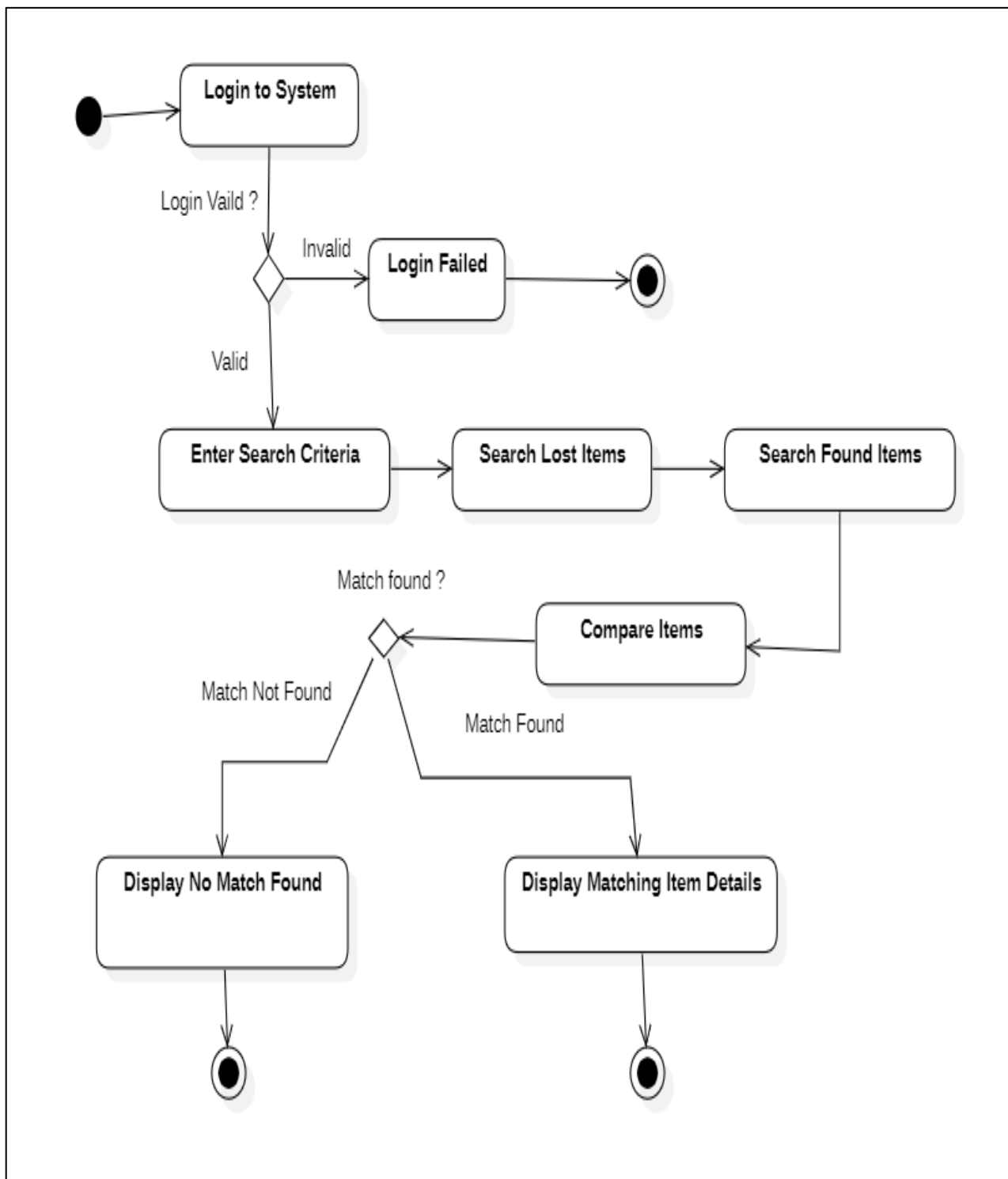
This activity diagram shows the steps followed when a user reports a lost item. The user logs in, enters the lost item details, and submits them to the system. The system validates the information, stores the lost item, and confirms the successful submission.



Module 4 :- Searching and Matching

➤ Description :-

This activity diagram explains how the system searches for lost and found items. The user enters search details, and the system looks through both lost and found records. The system then compares the items and shows the user whether a matching item is found or not.



E. Sequence Diagrams

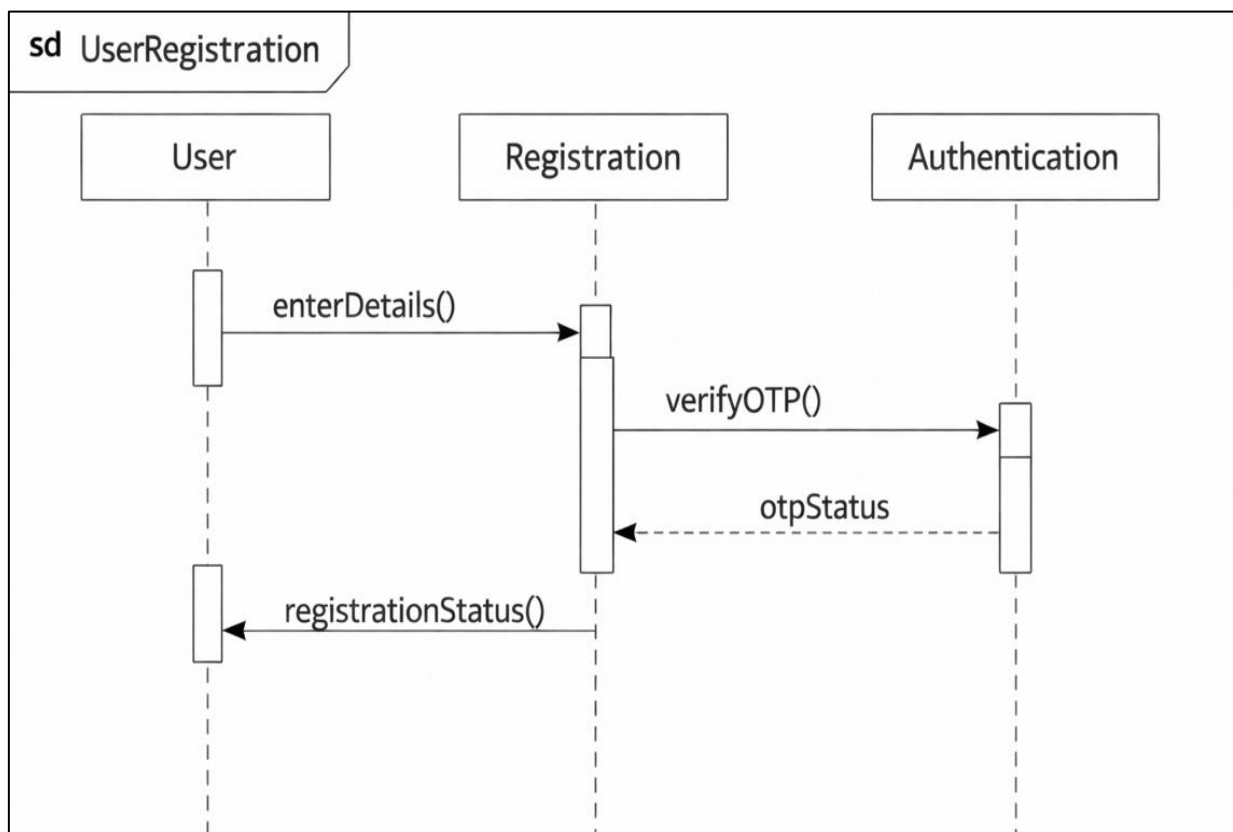
Module 1 :- User Management

➤ Description :-

This sequence diagram explains how a new user registers in the system. The user enters registration details, which are processed by the Registration class. The system verifies the information through Authentication and confirms whether the registration is successful. Finally, the result is sent back to the user.

➤ Lifelines :-

- User
- Registration
- Authentication



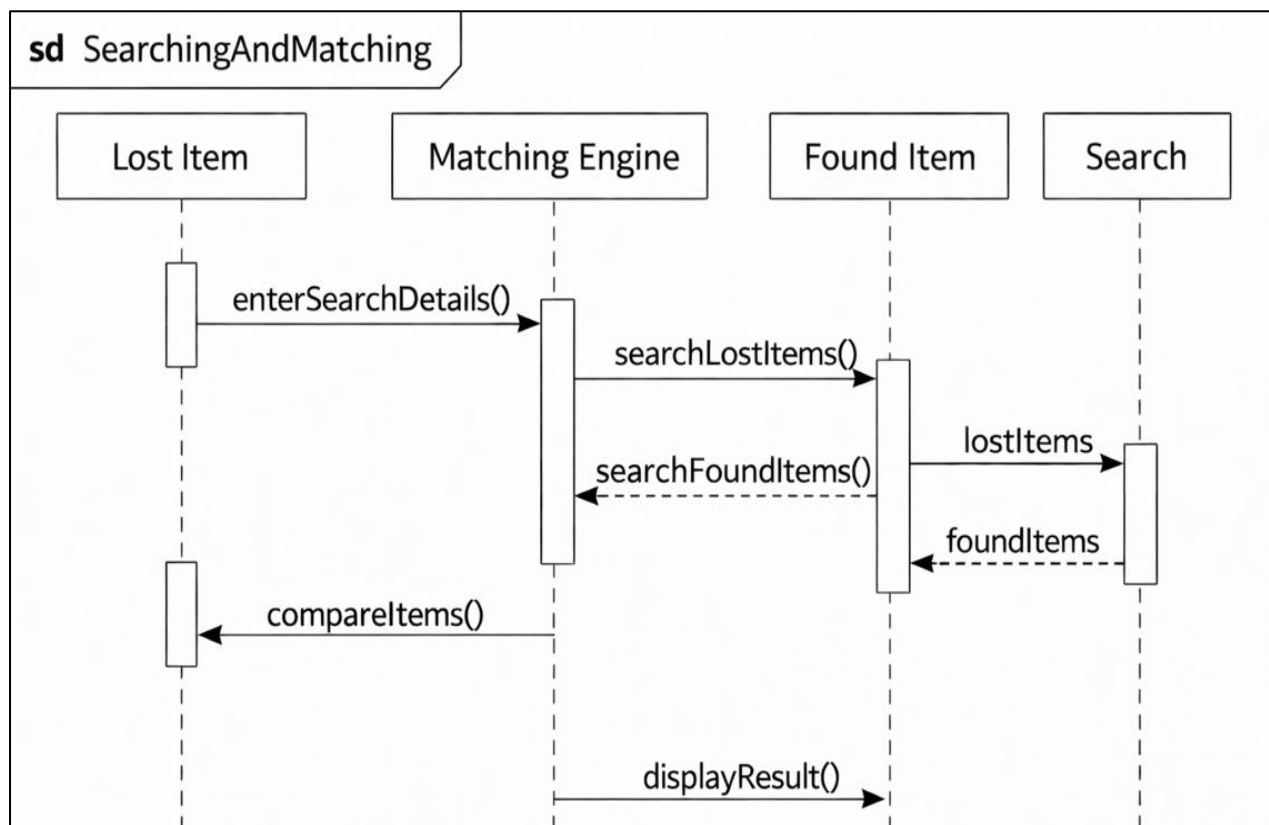
Module 4 :- Searching and Matching

➤ Description :-

This sequence diagram shows how the system searches and matches lost and found items. The Search class sends search details to Lost Item and Found Item classes. The Matching Engine compares both records to identify possible matches. Based on the result, the system determines whether a matching item exists.

➤ Lifelines :-

- Lost Item
- Matching Engine
- Found Item
- Search



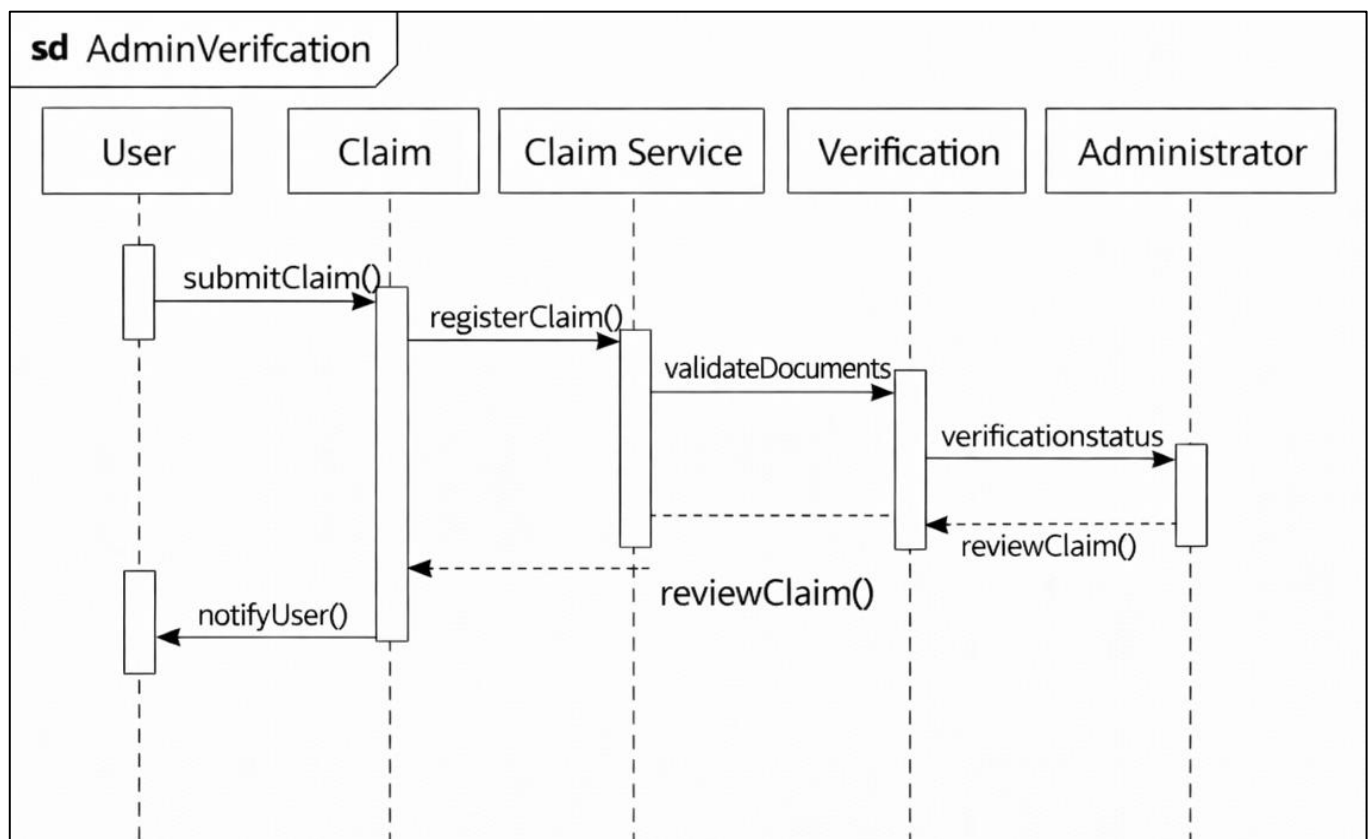
Module 5 :- Admin Verification

➤ Description :-

This sequence diagram describes how a claim is verified by the administrator. The user submits a claim, which is handled by the Claim Service and stored in the system. The administrator reviews the claim through the Verification class. After verification, the claim status is updated accordingly.

➤ Lifelines :-

- User
- Claim
- Claim Service
- Verification
- Administrator



F. Collaboration Diagrams

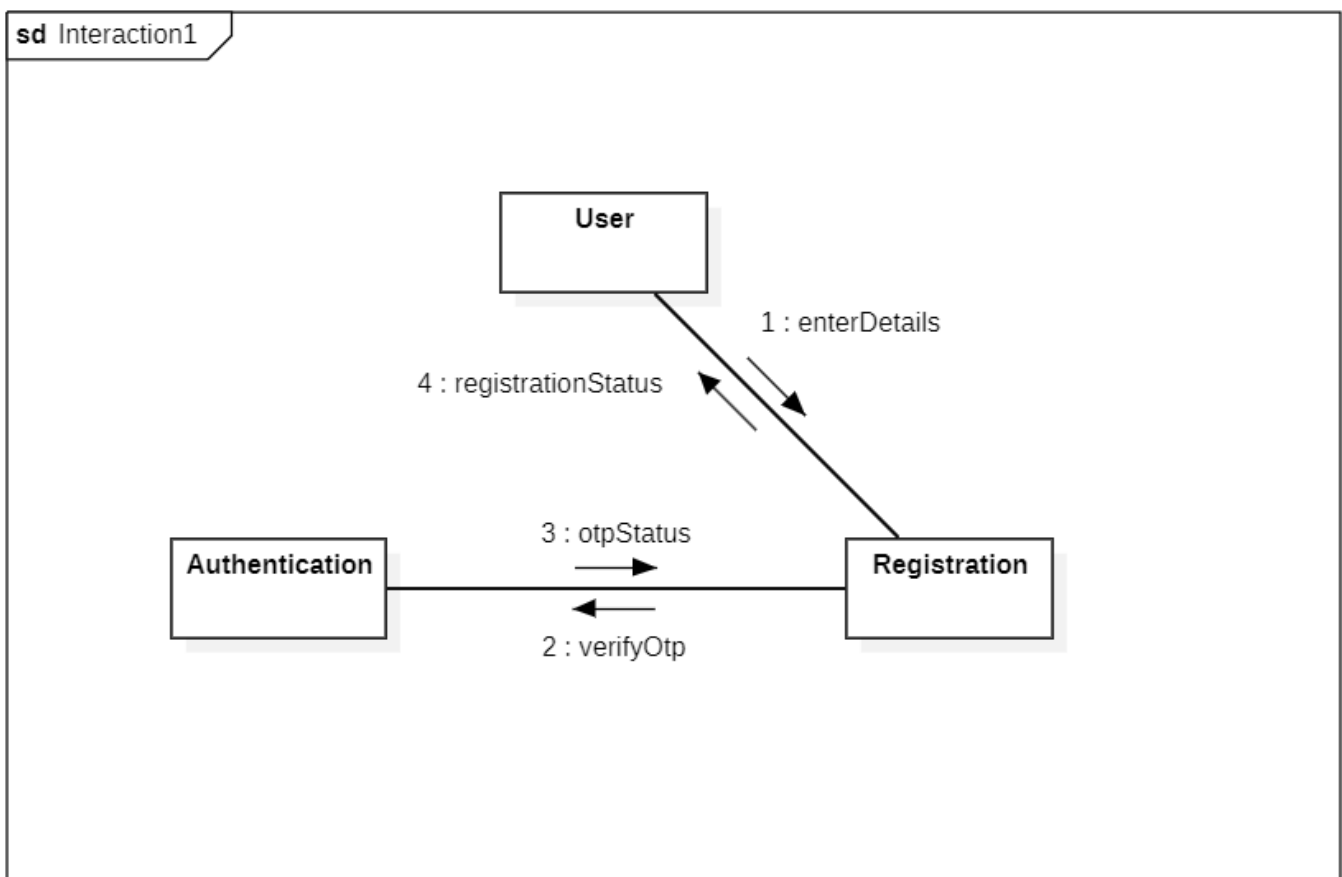
Module 1 :- User Management

➤ Description :-

This collaboration diagram shows how the User, Registration, and Authentication classes interact during the registration process. The user submits registration details to the Registration class, which verifies the information through the Authentication class. After OTP verification, the status is returned to the Registration class. Finally, the registration result is sent back to the user.

➤ Objects :-

- User
- Registration
- Authentication



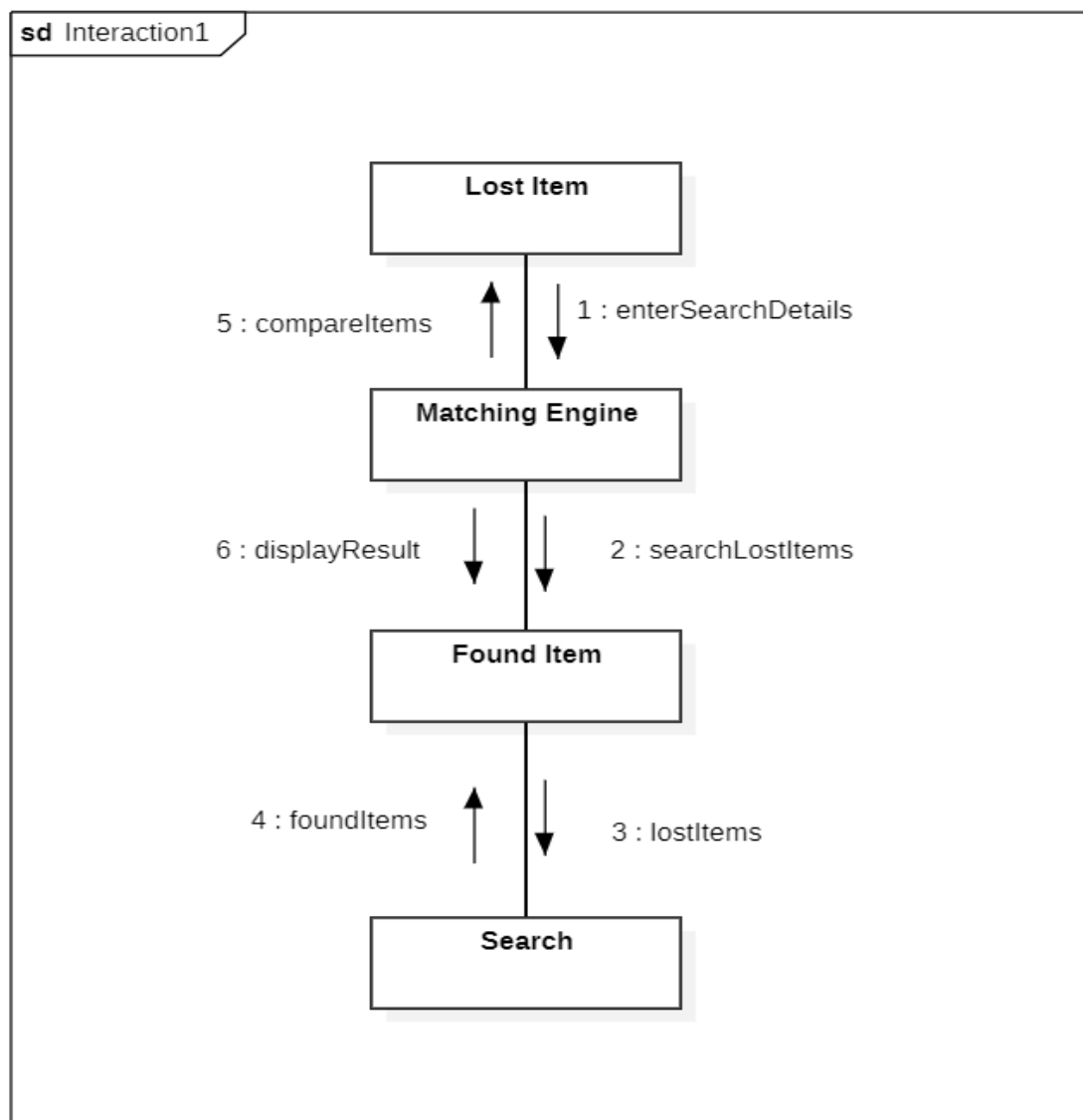
Module 4 :- Searching and Matching

➤ Description :-

This collaboration diagram illustrates the interaction between Lost Item, Matching Engine, Found Item, and Search classes. The search details are processed by the Matching Engine, which retrieves lost and found item records. The Matching Engine compares both sets of items to identify possible matches. Based on the comparison, the result is communicated within the system.

➤ Objects :-

- Lost Item
- Matching Engine
- Found Item
- Search



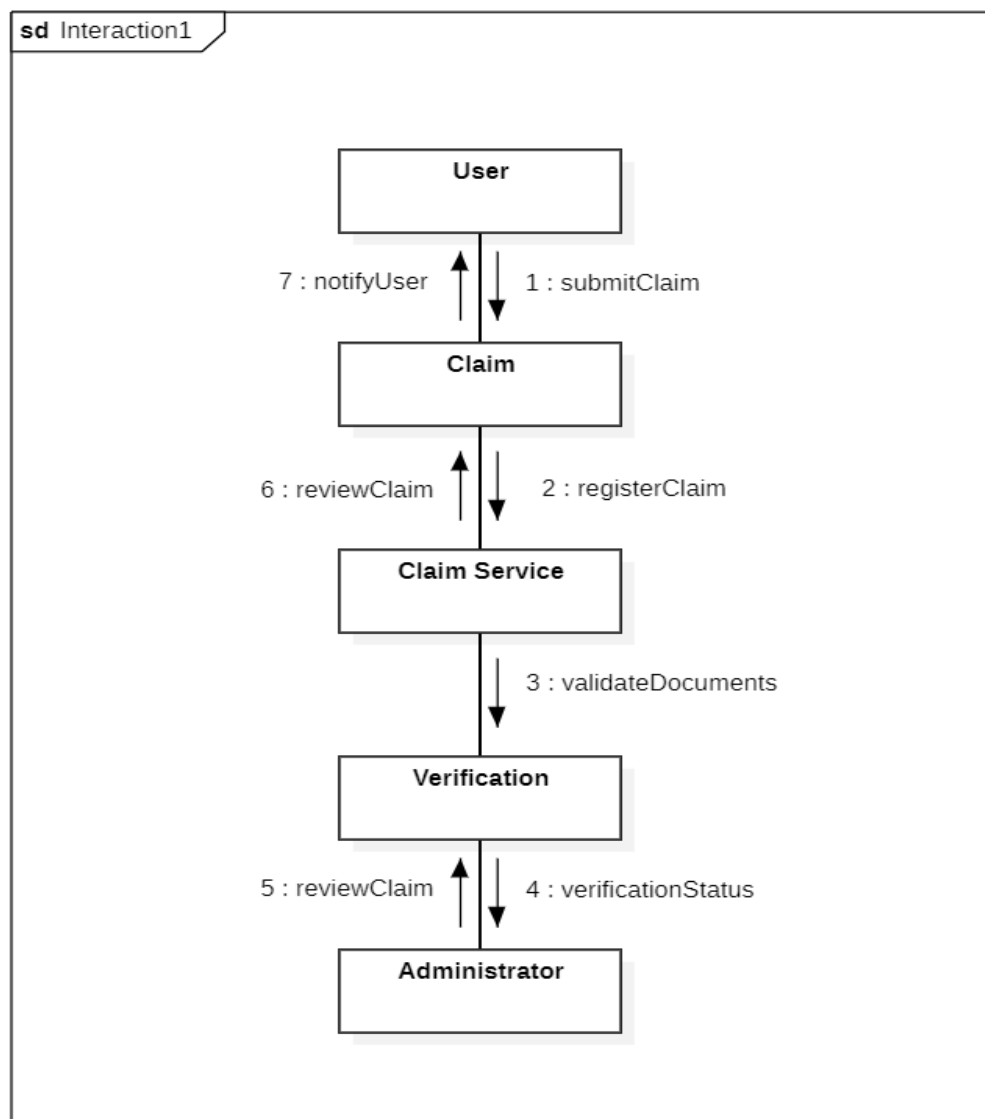
Module 5 :- Admin Verification

➤ Description :-

This collaboration diagram explains how the User, Claim, Claim Service, Verification, and Administrator classes interact during claim verification. The user submits a claim, which is validated by the Claim Service and Verification classes. The administrator reviews the verification status and makes a decision. Finally, the user is notified about the approval or rejection of the claim.

➤ Objects :-

- User
- Claim
- Claim Service
- Verification
- Administrator



3.2 ER Diagram

➤ Description :-

The ER diagram represents the data structure of the system, showing entities like User, Lost Item, Found Item, Claim, and Administrator and their relationships.

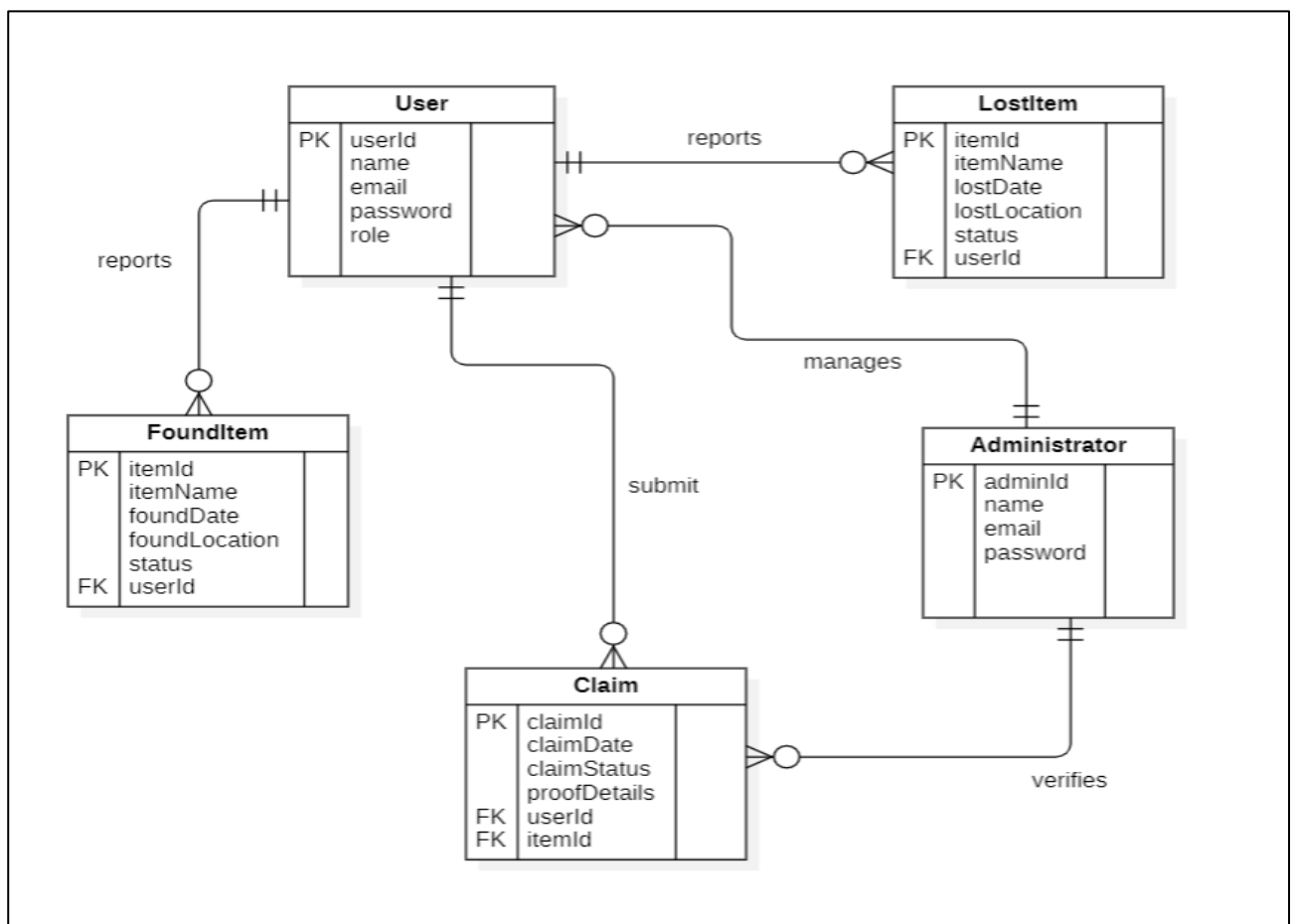
➤ Entities :-

- User
- Lost Item
- Found Item
- Claim
- Administrator

➤ Relationships :-

- A User reports Lost Item, Found Item(1 : Many)
- A User submits Claim (1 : Many)
- An Administrator manages users, verifies Claim (1 : Many)

➤ Diagram :-



3.3 DFD Diagram

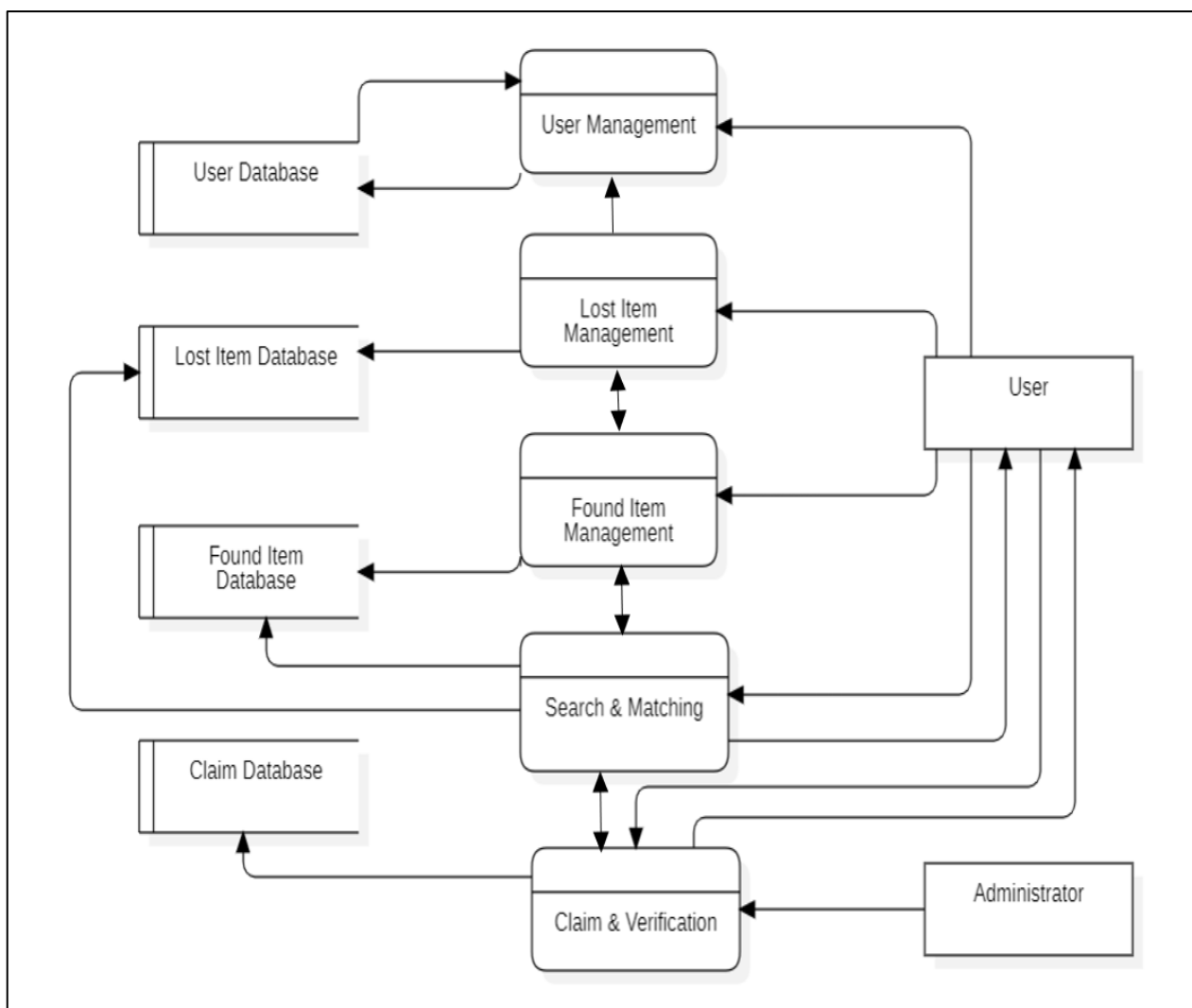
➤ Description :-

This Level-1 DFD shows a Lost and Found system where users register, report lost/found items, and search for matches. The system stores data in separate databases and performs matching. If a match is found, the claim is verified by the administrator and updated in the system.

➤ Process Description :-

- **User Management:** Handles user registration and login details.
- **Lost Item Management:** Records and stores details of lost items.
- **Found Item Management:** Records and stores details of found items.
- **Search & Matching:** Compares lost and found items to find matches.
- **Claim & Verification:** Verifies item claims and updates claim status.

➤ Diagram :-



4. External Interface Requirements

This chapter describes how the Digital Lost and Found System interacts with users and other external components. It explains the different interfaces provided by the system and how users communicate with it.

4.1 User Interfaces

This chapter describes how the Digital Lost and Found System interacts with users and other external components. It explains the different interfaces provided by the system and how users communicate with it.

- **Login and Registration Interface** :- It allows users to create an account and securely log into the system.
- **Item Reporting Interface** :- It enables users to report lost or found items by entering details such as item name, date, and location.
- **Claim and Verification Interface**: It allows users to submit claims for items and administrators to review and verify those claims.

4.2 Hardware Interfaces

This section describes the hardware components required to run the Digital Lost and Found System.

- **User Devices** :- The system can be accessed using desktop computers, laptops, or smartphones.

4.3 Software Interfaces

This section describes software components used in Digital Lost and Found System.

- **Operating System** :-The system is developed and tested on Windows 10 and Windows 11 operating system.
- **Database System** :- The system uses MySQL Database to store user details, lost items, found items, and claim information.
- **Development Platform and Browser** :-The application is developed using a web based platform and runs on Google Chrome browser for testing and execution.

4.4 Communication Interfaces

This section describes how the Digital Lost and Found System communicates between users and the server.

- **Client–Server Communication** :- The system follows client-server architecture, through a web browser and data is processed on the server.
- **Database Communication** :- The application connects to the MySQL database to store and request information.

5. Non Functional Requirements

This chapter describes the non-functional requirements of the Digital Lost and Found System. These requirements define how the system should perform, its quality standards, and operational constraints beyond basic functionality.

5.1 Performance Requirements

This section describes how efficiently the system should perform under normal usage conditions.

- **Response Time :-** The system should respond to user actions such as login, search, and item submission within a few seconds.
- **Concurrent Users :-** The system should support multiple users accessing and using the application at the same time without performance degradation.

5.2 Safety Requirements

This section describes the safety measures to ensure that system data and user information are protected from accidental loss or misuse.

- **Data Backup :-** The system should regularly maintain database backups to prevent loss of important user and item information.
- **Access Control :-** Only authorized administrators should have access to claim verification and system management features.
- **Error Handling :-** The system should handle errors properly and display clear messages without crashing or exposing sensitive information.

5.3 Security Requirements

This section describes the security measures implemented in the Digital Lost and Found System to protect user data and system information.

- **User Authentication :-** The system requires users to log in using valid credentials before accessing system features.
- **Data Protection :-** Sensitive information such as passwords should be securely stored in the database and protected from unauthorized access.

5.4 Software Quality Attributes

This section describes the quality characteristics that the Digital Lost and Found System should maintain.

- **Usability :-** The system should be easy to use with a simple and clear interface so that users can report and search items without difficulty.

- **Reliability :-** The system should function correctly without frequent failures and should maintain accurate data records.

5.5 Business Policies / Rules

This section describes the rules and policies that must be followed in the Digital Lost and Found System.

- **Claim Verification Rule :-** A claim must be verified and approved by the administrator before the item status is updated as returned.
- **Administrative Authority Rule :-** Only administrators have the authority to approve or reject claims.

6. Testing

6.1 White Box Testing :-

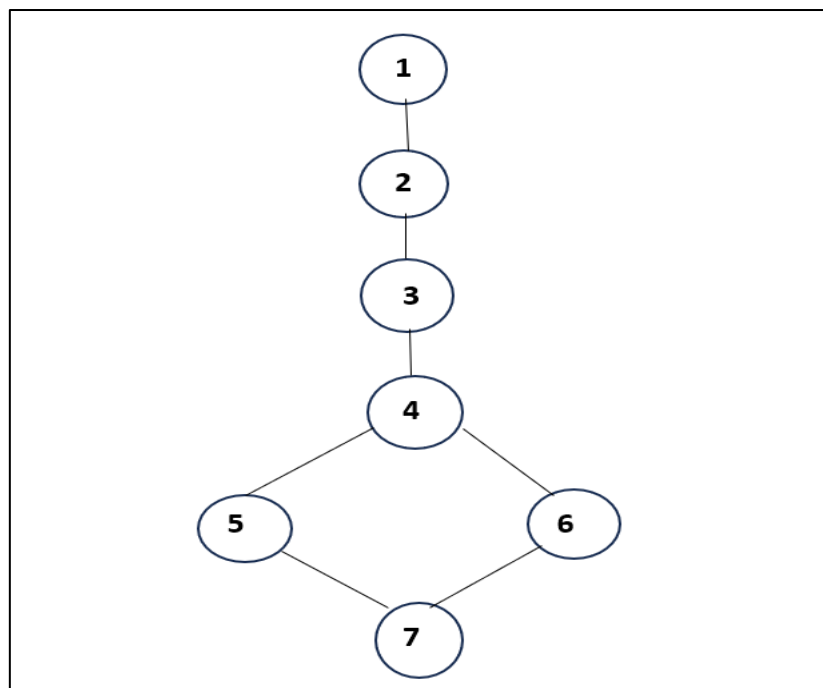
White-Box Testing checks the internal logic of the program. It analyzes the source code structure, control flow, and independent execution paths.

Sample Code 1 :- Module 1 – User Management

```
def validate_login(username, password):  
    correct_username = "user"  
    correct_password = "12345"  
    if username == correct_username and password == correct_password:  
        return True  
    else:  
        return False  
user = input("Enter username: ")  
pwd = input("Enter password: ")  
if validate_login(user, pwd):  
    print("Login Successful")  
else:  
    print("Invalid Credentials")
```

6.1.1 Flow Graph Notation

A Flow Graph uses nodes to represent processing statements and edges to map the logical paths between them. The flow graph shows how the program moves from login input to validation and output.



6.1.2 Cyclomatic Complexity (Using Graph Method)

Here $N = 7$, $E = 7$

Where E = Number of edges

N = Number of nodes

$$V(G) = E - N + 2$$

$$V(G) = 7 - 7 + 2 = 2$$

So,

Cyclomatic Complexity = 2

Meaning there are 2 independent paths.

6.1.3 Independent Program Paths

As Cyclomatic Complexity = 2

That means: we must identify 2 independent paths.

Path 1 – Valid Login

1, 2, 3, 4, 5, 7

Path 2 – Invalid Login

1, 2, 3, 4, 6, 7

Independent Paths and Test :-

Path No	Independent Path	Test Input	Expected Output
P1	1,2,3,4,5,7	username="user", password="12345"	Login Successful
P2	1,2,3,4,6,7	username="user", password="1111"	Invalid Credentials

6.1.4 Implement Unit Tests (Execution Phase)

Convert the above table into Python unittest.

Unit test Code :-

```
import unittest
def validate_login(username, password):
    correct_username = "user"
    correct_password = "12345"

    if username == correct_username and password == correct_password:
        return "Login Successful"
    else:
        return "Invalid Credentials"
class TestLogin(unittest.TestCase):
    def test_valid_login(self):
        self.assertEqual(validate_login("user","12345"),"Login Successful")
    def test_invalid_login(self):
        self.assertEqual(validate_login("user","0000"),"Invalid Credentials")
if __name__ == "__main__":
    unittest.main()
```

Unit Test Output :-

```
..
-----
Ran 2 tests in 0.000s

OK

...Program finished with exit code 0
Press ENTER to exit console.█
```

6.2 Black-Box Testing

Functional behavior verified without analyzing internal code.

Sample Code 1 :- Module 1 – User Management

```
def validate_login(username, password):
    correct_username = "user"
    correct_password = "12345"
    if username == correct_username and password == correct_password:
        return True
    else:
        return False
user = input("Enter username: ")
pwd = input("Enter password: ")
if validate_login(user, pwd):
    print("Login Successful")
else:
    print("Invalid Credentials")
```

Step-1 Functional Requirements (Login Module)

System should:

- Accept username and password as input
- Validate login credentials
- Allow login only if credentials are correct
- Reject login if credentials are incorrect

Step-2: Black Box Testing Techniques

2.1 Equivalence Partitioning:

Inputs are divided into valid and invalid groups.

For User Input :-

Class	Condition	Type
EC1	username="user"	Valid
EC2	username!="user"	Invalid
EC3	username empty	Invalid

For Password Input :-

Class	Condition	Type
EC4	password="12345"	Valid
EC5	password!="12345"	Invalid
EC6	password empty	Invalid

For Login Credential Cases :-

Class	Condition	Result
EC7	Correct username + correct password	Login Successful
EC8	Correct username + wrong password	Invalid Credentials
EC9	Wrong username + correct password	Invalid Credentials
EC10	Wrong username + wrong password	Invalid Credentials

2.2 Boundary Value Analysis

Username Boundary

Test	Username	Type	Expected
BB1	""	Below limit	Invalid Credentials
BB2	"u"	Minimum	Invalid Credentials
BB3	"user"	Exact	Depends on password

Password Boundary

Test	Password	Type	Expected
BB4	""	Below limit	Invalid Credentials
BB5	"1"	Minimum	Invalid Credentials
BB6	"12345"	Exact	Depends on username

Combined Test Cases

Test	Username	Password	Expected
BB7	""	"12345"	Invalid Credentials
BB8	"user"	""	Invalid Credentials
BB9	"u"	"1"	Invalid Credentials
BB10	"user"	"12345"	Login Successful

Run the Program manually or using function calls.

```
def validate_login(username, password):
    correct_username = "admin"
    correct_password = "12345"
    if username == correct_username and password == correct_password:
        return "Login Successful"
    else:
        return "Invalid Credentials"

print("Test Case 1: Correct username and password")
print(validate_login("admin", "12345"))
print("Test Case 2: Correct username, wrong password")
print(validate_login("admin", "11111"))
print("Test Case 3: Wrong username, correct password")
print(validate_login("user", "12345"))
print("Test Case 4: Wrong username and wrong password")
print(validate_login("user", "00000"))
print("Test Case 5: Empty username")
print(validate_login("", "12345"))
print("Test Case 6: Empty password")
print(validate_login("admin", ""))
```

Output :-

```
Test Case 1: Correct username and password
Login Successful
Test Case 2: Correct username, wrong password
Invalid Credentials
Test Case 3: Wrong username, correct password
Invalid Credentials
Test Case 4: Wrong username and wrong password
Invalid Credentials
Test Case 5: Empty username
Invalid Credentials
Test Case 6: Empty password
Invalid Credentials
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

The Output is matched to all the above test cases mentioned in table, then Black Box testing is successful.

6.1 White Box Testing :-

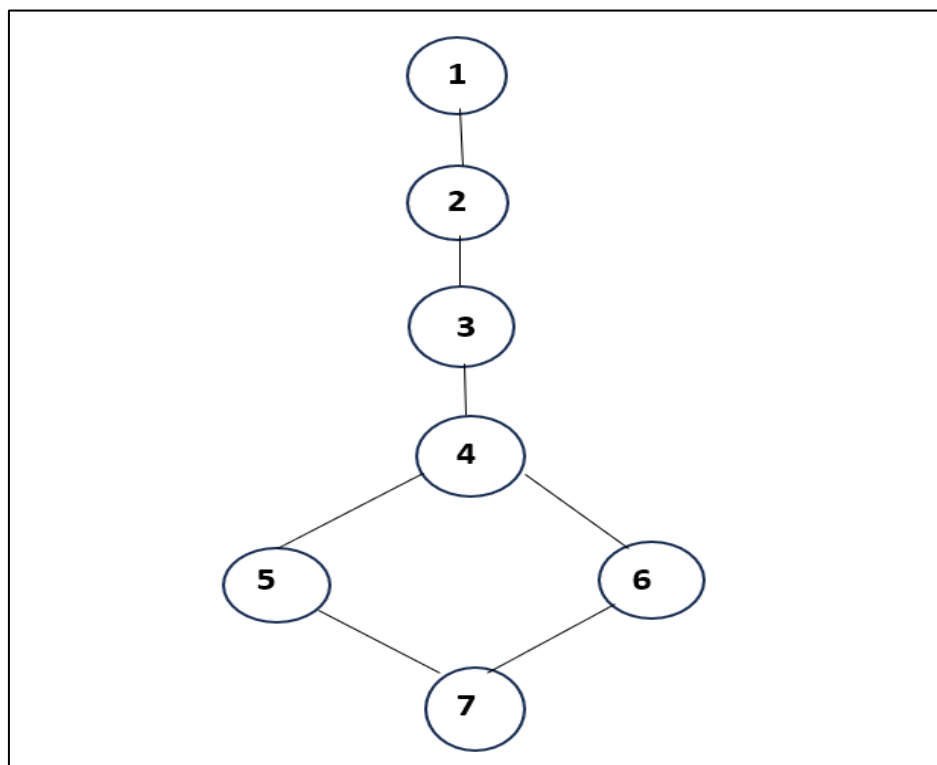
White-Box Testing checks the internal logic of the program. It analyzes the source code structure, control flow, and independent execution paths.

Sample Code 2 :- Module 3 – Item Search Validation

```
def search_item(item_name):  
    available_items = ["laptop", "phone", "bag", "wallet"]  
    if item_name in available_items:  
        return "Item Found"  
    else:  
        return "Item Not Found"  
  
item = input("Enter item name: ")  
result = search_item(item)  
print(result)
```

6.1.1 Flow Graph Notation

A Flow Graph uses nodes to represent processing statements and edges to map the logical paths between them. The flow graph shows how the program moves from login input to validation and output.



6.1.2 Cyclomatic Complexity (Using Graph Method)

Here $N = 7$, $E = 7$

Where E = Number of edges

N = Number of nodes

$$V(G) = E - N + 2$$

$$V(G) = 7 - 7 + 2 = 2$$

So,

Cyclomatic Complexity = 2

Meaning there are 2 independent paths.

6.1.3 Independent Program Paths

As Cyclomatic Complexity = 2

That means: we must identify 2 independent paths.

Path 1 – Valid Login

1, 2, 3, 4, 5, 7

Path 2 – Invalid Login

1, 2, 3, 4, 6, 7

Independent Paths and Test :-

Path No	Independent Path (Node Sequence)	Test Input	Expected Output
P1	1,2,3,4,5,7	"laptop"	Item Found
P2	1,2,3,4,6,7	"watch"	Item Not Found

6.1.4 Implement Unit Tests (Execution Phase)

Convert the above table into Python unittest.

Unit test Code :-

```
import unittest

def search_item(item_name):

    available_items = ["laptop", "phone", "bag", "wallet"]

    if item_name in available_items:
        return "Item Found"
    else:
        return "Item Not Found"

class TestSearch(unittest.TestCase):

    def test_item_found(self):
        self.assertEqual(search_item("laptop"), "Item Found")

    def test_item_not_found(self):
        self.assertEqual(search_item("watch"), "Item Not Found")

if __name__ == "__main__":
    unittest.main()
```

Unit test Output :-

```
..
-----
Ran 2 tests in 0.000s

OK

...Program finished with exit code 0
Press ENTER to exit console.█
```

6.2 Black-Box Testing

Functional behavior verified without analyzing internal code.

Sample Code 2 :- Module 3 – Item Search Validation

```
def search_item(item_name):
    available_items = ["laptop", "phone", "bag", "wallet"]
    if item_name in available_items:
        return "Item Found"
    else:
        return "Item Not Found"

item = input("Enter item name: ")
result = search_item(item)
print(result)
```

Step-1 Functional Requirements

System should:

- System shall accept item name as input
- System shall compare item name with stored items
- If item exists in the list, return Item Found
- If item does not exist, return Item Not Found

Step-2: Black Box Testing Techniques

2.1 Equivalence Partitioning:

Inputs are divided into valid and invalid groups.

Item Name Input

Class	Condition	Valid / Invalid
EC1	Item exists in list	Valid
EC2	Item not in list	Invalid
EC3	Item name empty	Invalid

Item Search Cases

Class	Condition	Result
EC4	Valid item name	Item Found
EC5	Invalid item name	Item Not Found

2.2 Boundary Value Analysis

Item Name Boundary

BB No	Test Value	Type	Expected Output
BB1	""	Empty input	Item Not Found
BB2	"a"	Minimum length	Item Not Found
BB3	"laptop"	Exact match	Item Found

Item Cases Boundary

BB No	Item Name	Type	Expected Output
BB4	""	Empty value	Item Not Found
BB5	"bag"	Valid item	Item Found
BB6	"watch"	Invalid item	Item Not Found

Run the Program manually or using function calls.

```
def search_item(item_name):
    available_items = ["laptop", "phone", "bag", "wallet"]
    if item_name in available_items:
        return "Item Found"
    else:
        return "Item Not Found"

print("Test Case 1: Empty input")
print(search_item(""))
print("Test Case 2: Valid item")
print(search_item("bag"))
print("Test Case 3: Invalid item")
print(search_item("watch"))
print("Test Case 4: Another valid item")
print(search_item("laptop"))
```

Output :-

```
Test Case 1: Empty input
Item Not Found
Test Case 2: Valid item
Item Found
Test Case 3: Invalid item
Item Not Found
Test Case 4: Another valid item
Item Found

...Program finished with exit code 0
Press ENTER to exit console.
```

The Output is matched to all the above test cases mentioned in table, then Black Box testing is successful.

7. Conclusion :-

- The Digital Lost and Found System provides an efficient platform for users to report and search for lost or found items in a structured manner.
- The system simplifies the recovery process by allowing users to submit item details and track the status of their reports online.
- Administrator verification helps ensure the authenticity of claims and improves the reliability and security of the system.
- Overall, the system reduces manual effort and helps users recover lost items quickly and efficiently.

8. Reference :-

- Online tutorials and documentation related to UML diagrams that helped in designing use case, class, sequence, and component diagrams.
- Online resources such as diagrams.net (draw.io) and StarUML tutorials, which were used to create system diagrams and design the architecture of the project.

9. Acknowledgement :-

- Through this project, we learned how to prepare a Software Requirement Specification (SRS) document and clearly define the requirements of a software system.
- We gained practical experience in designing UML diagrams such as use case diagrams, class diagrams, activity diagrams, and component diagrams to represent system behavior and structure.

DESIGN

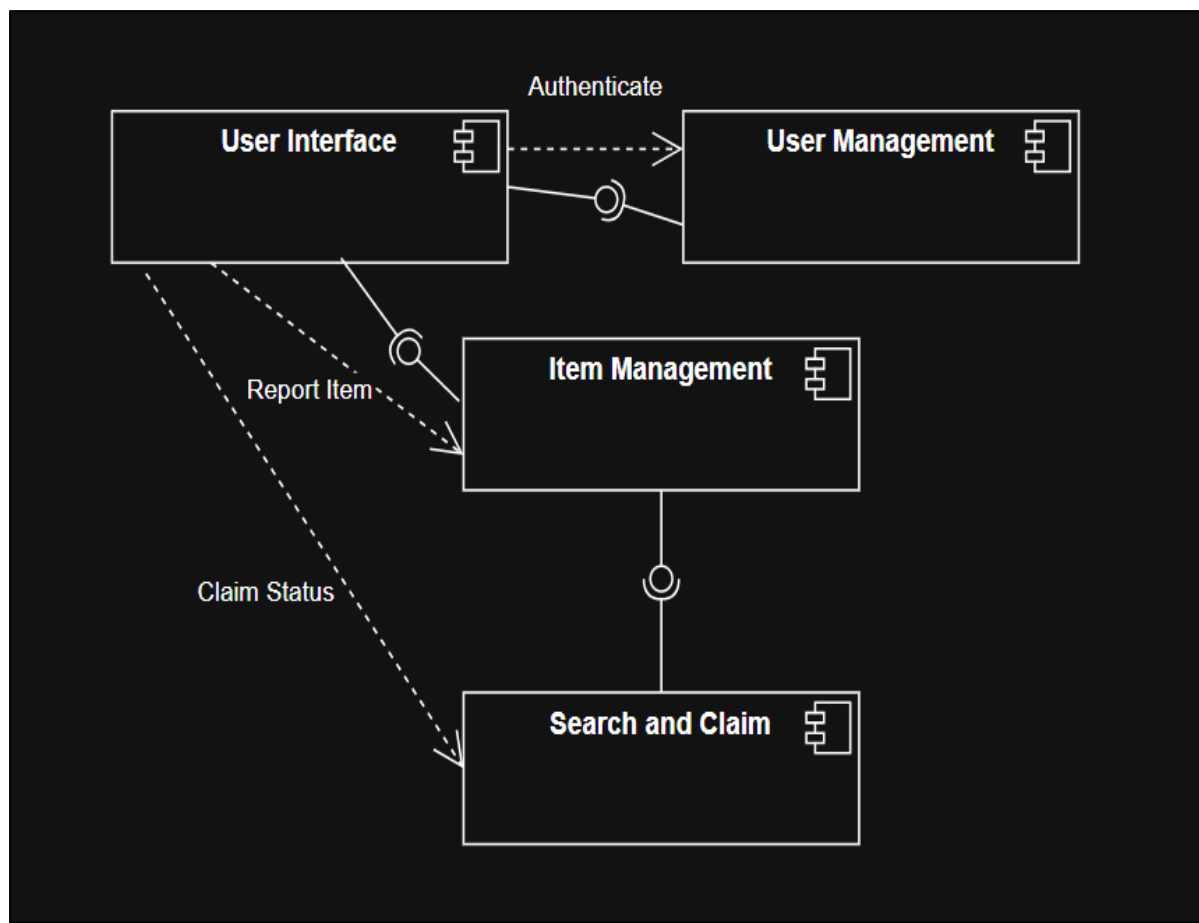
ARCHITECTURE

Design Phase :-

The design phase of the Digital Lost and Found System focused on planning the system structure and how different modules interact with each other. UML diagrams such as use case, class, and component diagrams were created to represent the system design. This phase helped transform the system requirements into a clear blueprint for implementation.

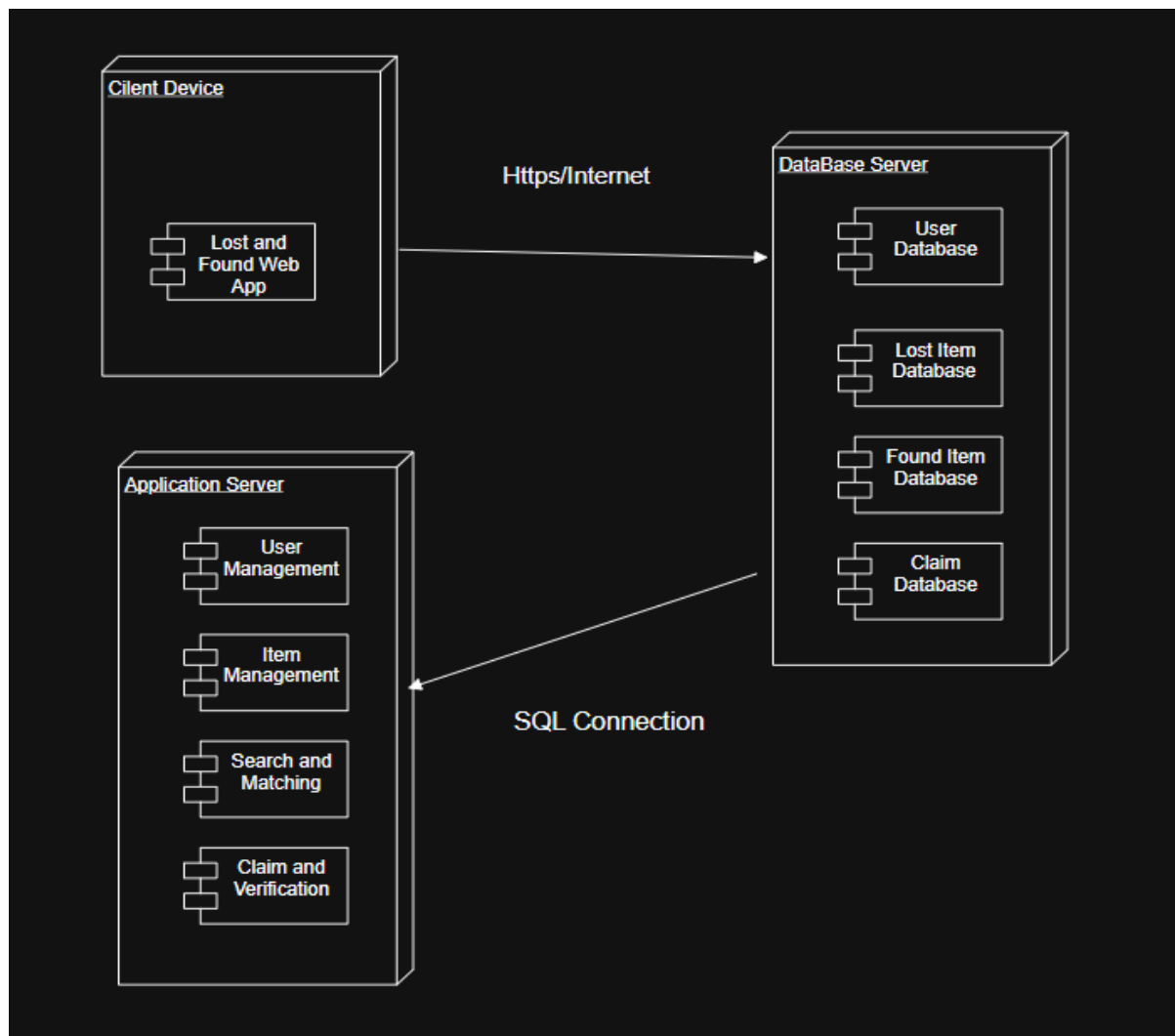
Component Diagram :-

The component diagram shows the main parts of the Digital Lost and Found System and how they interact with each other. It represents components such as the user interface, user management, item management, and search or claim modules. This diagram helps in understanding how different modules communicate through interfaces.



Deployment Diagram :-

The deployment diagram illustrates how the system is deployed in a real environment. It shows the connection between the client device, application server, and database server. This diagram helps explain where the software components are installed and how they communicate over the network.



METRICS FOR **ANALYSIS MODEL**

FUNCTION POINT ANALYSIS

1. Product Metrics:

Product metrics measures the quality and the functionality of the software product. For the Digital Lost and Found System, these metrics help evaluate:

- System functionality
- Complexity
- Maintainability
- Performance

2. Analysis Model Metrics:

Analysis model metrics estimate the size and complexity of the system during requirement analysis.

For the Digital Lost and Found System, the main metrics include:

1) External Inputs (EIs): 5

Inputs are data entered by the user.

Examples:

- User Registration details
- Login credentials
- Lost item report submission
- Found item report submission
- Claim request submission

2) External Outputs (EOs): 5

Outputs are data sent from the system to the user.

Examples:

- Registration confirmation
- Login success/error message
- Lost/Found item display results
- Claim status (Approved/Rejected)
- Notification messages

3) External Inquiries (EQs): 3

These are interactive inputs requiring immediate response.

Examples:

- Search for lost items
- Search for found items
- View item details

4) Internal Logical Files (ILFs): 4

These are internal data stores maintained by the system.

Examples:

- User database
- Lost items database
- Found items database
- Claims database

5) External Interface Files (EIFs): 1

These are external systems interacting with the application.

Examples:

- Email/Notification service

3. Function Point Calculation:

Component	Count	Weight	Total
External Inputs (EI)	5	4	20
External Outputs (EO)	5	5	25
External Inquiries (EQ)	3	4	12
Internal Logical Files (ILF)	4	10	40
External Interface Files (EIF)	1	7	7

Total Function Points (FP) = 104

4. Complexity Adjustment Factor (CAF):

CAF Formula:

$$CAF = 0.65 + (0.01 \times \Sigma(Fi))$$

Value Adjustment Factors:

1. **Reliable backup and recovery required?** – Score: 4
– User data and item reports must be securely stored and recoverable.
2. **Specialized data communication required?** – Score: 3
– The system communicates with database and notification services.
3. **Distributed processing functions present?** – Score: 2
– Web application connected with database.
4. **Is performance critical?** – Score: 4
– Fast search and matching of lost and found items is required.
5. **Runs in heavily used environment?** – Score: 3
– Multiple users may access the system simultaneously.

6. **Online data entry required?** – Score: 5
– Users report lost/found items and submit claims online.
7. **Multi-screen online data entry required?** – Score: 3
– Includes reporting items, searching, and claim submission.
8. **ILFs updated online?** – Score: 5
– User, lost item, found item, and claim data updated in real time.
9. **Inputs/outputs/files complex?** – Score: 3
– Moderate complexity in item reports and search results.
10. **Internal processing complex?** – Score: 4
– Matching lost and found items requires processing logic.
11. **Code designed for reuse?** – Score: 3
– Modules like login, search, and reporting can be reused.
12. **Conversion and installation included?** – Score: 1
– Minimal setup required.
13. **Multiple installations required?** – Score: 2
– Can be used in colleges, hostels, offices.
14. **Designed for change and ease of use?** – Score: 4
– User-friendly and scalable system.

Calculation:

$$\Sigma(F_i) = 4 + 3 + 2 + 4 + 3 + 5 + 3 + 5 + 3 + 4 + 3 + 1 + 2 + 4 = 46$$

$$CAF = 0.65 + (0.01 \times 46)$$

$$CAF = 0.65 + 0.46 = 1.11$$

5. Final Function Point:

$$FP = UFP \times CAF$$

$$FP = 104 \times 1.11$$

$$FP = 115.44$$

Therefore,

Function Point (FP) = 115.44

Estimated Development Cost:

Assume: If 1 FP assumes to be 10 hours of development, then

$$115.44 \times 10 = 1154.4 \text{ hours}$$

If Developer Cost = ₹1000 per hour

Final Estimated Cost = ₹11,54,400 /-

EMPIRICAL ESTIMATION MODEL (COCOMO MODEL)

COCOMO MODEL:

The COCOMO (Constructive Cost Model) is a standard method used in software engineering to estimate the effort, cost, and schedule required for a software project. Introduced by Barry Boehm in 1981, it relies on historical data and mathematical formulas to predict project outcomes based primarily on the size of the software.

There are 2 Types of Models:

- **Basic COCOMO Model**
- **Intermediate COCOMO Model**

1. Basic COCOMO Model:

The Basic COCOMO can be used for quick and slightly rough calculations of Software Costs.

The following formulas are used to calculate effort and time estimates:

- $\text{Effort (E)} = A \times (\text{KLOC})^B$
- $\text{Time (T)} = C \times (\text{Effort})^D$
- $\text{Persons required} = \text{Effort} / \text{Time}$

Constant Values Table :-

Software Project	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Project Details :-

The Digital Lost and Found System is a Small Organic Project, with an estimated size of 8 KLOC.

- $A = 2.4$
- $B = 1.05$
- $C = 2.5$
- $D = 0.38$

Calculations :-

Effort:

$$\text{Effort} = 2.4 \times (8)^{1.05}$$

$$\approx 2.4 \times 8.57$$

$$\approx 20.57 \text{ person-months}$$

Time:

$$\text{Time} = 2.5 \times (20.57)^{0.38}$$

$$\approx 2.5 \times 3.15$$

$$\approx 7.87 \text{ months}$$

Persons Required:

$$\text{Persons} = 20.57 / 7.87$$

$$\approx 2.61 \approx 3 \text{ persons}$$

Basic COCOMO Result

- KLOC = 8
- Effort = 20.57 PM
- Time = 7.87 months
- Team Size = 3 persons

2. Intermediate COCOMO Model:

The Intermediate COCOMO model estimates software effort by considering both size (KLOC) and cost drivers. It uses an Effort Adjustment Factor (EAF) to account for factors like product complexity, team capability, and tools.

Cost Drivers**Product Attributes**

- **Required software reliability**
Since user data and item details must be accurate.
Rating: High = 1.15
- **Size of application database**
Moderate database storing users, items, and claims.
Rating: Nominal = 1.00
- **Product complexity**
Includes login, search, matching, claim verification.
Rating: High = 1.15

Hardware Attributes

- **Run-time performance constraints**
No real-time processing required
Rating: Nominal = 1.00

- **Memory constraints**
No strict limitations
Rating: Nominal = 1.00
- **Virtual machine volatility**
Stable environment
Rating: Nominal = 1.00
- **Turnaround time**
No strict deadlines
Rating: Nominal = 1.00

Personnel Attributes

- **Analyst capability**
Student-level project
Rating: Nominal = 1.00
- **Software engineering capability**
Average
Rating: Nominal = 1.00
- **Application experience**
Limited domain experience
Rating: Low = 1.17
- **Virtual machine experience**
Normal
Rating: Nominal = 1.00
- **Programming language experience**
Familiar with Java/Web
Rating: Nominal = 1.00

Project Attributes

- **Use of software tools**
Good tools used (IDE, DB, diagrams)
Rating: High = 0.91
- **Software engineering methods**
SRS, UML, testing followed
Rating: High = 0.91
- **Development schedule**
No strict pressure
Rating: Nominal = 1.00

Tabular representation :

Cost Driver No	Very Low	Low	Nominal	High	Very High	Extra High
1				1.15		
2			1.00			
3				1.15		
4			1.00			
5			1.00			
6			1.00			
7			1.00			
8			1.00			
9			1.00			
10		1.17				
11			1.00			
12			1.00			
13				0.91		
14				0.91		
15			1.00			

EAF Calculation

EAF = Product of all cost drivers

$$\text{EAF} = 1.15 \times 1.00 \times 1.15 \times 1.00 \times 1.00 \times 1.00 \times 1.00 \times 1.00 \times 1.00 \times 1.17 \times 1.00 \times 1.00 \times 0.91 \times 0.91 \times 1.00$$

$$\text{EAF} \approx 1.28$$

Intermediate Calculations

Effort:

$$\text{Effort} = 2.4 \times (8)^{1.05} \times 1.28$$

$$\approx 20.57 \times 1.28$$

$$\approx 26.33 \text{ person-months}$$

Time:

$$\text{Time} = 2.5 \times (26.33)^{0.38}$$

$$\approx 2.5 \times 3.32$$

$$\approx 8.30 \text{ months}$$

Persons Required:

$$\text{Persons} = 26.33 / 8.30$$

$$\approx 3.17 \approx 3 \text{ persons}$$

Final Result (Intermediate COCOMO)

- Effort = 26.33 PM
- Development Time = 8.30 months
- Team Size = 3 persons

